II. Physikalisches Anfängerpraktikum

UNIVERSELLE HEIDELBEERE
EIGENTLICH EINE BROOMBEERE

Protocol for the Experiment
X-Ray

(Experiment 255)

Author: Finn Zeumer
Experimental Partner: Annika Künstle
Tutor: Friso Grace

Date of the experiment: 23.06.2026
Hand-in Date: June 27, 2026

FEZ

Preface and Notes

Use of External Tools

For the linguistic drafting and correction of the Introduction (Kapitel **??**) and the Execution (Kapitel 3.), the AI *Lumo AI* as well as *Duden Mentor* were employed. However, the substantive evaluation and the actual creation of the document were carried out entirely without the assistance of Artificial Intelligence, unless explicitly marked otherwise. [Dud26, Pro26]

All graphics contained in the document that lack a specific reference were independently created or edited in Inkscape, or are plots generated in Python. Both the Python code and the LaTeX document were prepared in the Visual Studio Code (VS Code) development environment.

Own Code and Transparency

To carry out the evaluations, a custom Python library was developed. The entire source code - both the Python scripts and the LaTeX sources - is self-created and publicly available online.

The entire project structure can be found on my GitHub. Here, both the Python code and the LaTeX document code are documented. [Fin26a]

The Python functions used are specifically identifiable on GitHub under the name "Papulator" [Fin26b]

Use of University Materials

This document is based on the script by Jens Wagner (as of April 27, 2026). Graphics from this script were adopted for the Introduction. These excerpts are transparently marked; the figures used always originate from the chapters of the script that share the same experiment number and experiment name as in the present document. In this experiment, the graphics from *Experiment 255 X-Ray* were used. [Wag26a]

Table of Content

1. Introduction

1.1 Task and Motivation

In this experiment, the generation and analysis of X-ray radiation are investigated. The aim is, on the one hand, to determine fundamental constants of physics experimentally and, on the other hand, to analyze the structure of crystals using diffraction methods. Specifically, Planck's quantum of action h is to be determined from the short-wavelength end of the bremsstrahlung spectrum as well as from the threshold voltage at characteristic wavelength. Furthermore, the wavelengths of the characteristic K_α and K_β lines of a molybdenum anode are to be determined. By using lithium fluoride (LiF) and sodium chloride (NaCl) as diffraction crystals, the connection between crystal lattice structure and macroscopic quantities such as the Avogadro number N_A can also be established.

1.2 Physical Basis

1.2.1 Generation of X-ray Radiation

The generation of X-ray radiation in this experiment takes place via an X-ray tube with a molybdenum anode and a heated filament cathode. Inside an evacuated glass vessel, electrons are accelerated by an electric field between the cathode and the anode. The kinetic energy E of the electrons results from the applied accelerating voltage U:

$$E = eU \tag{1.1}$$

where e represents the elementary charge. Upon impact with the anode, the electrons are decelerated in the Coulomb field of the atomic nuclei. This braking process leads to the emission of a continuous spectrum of electromagnetic radiation, known as the bremsstrahlung spectrum.

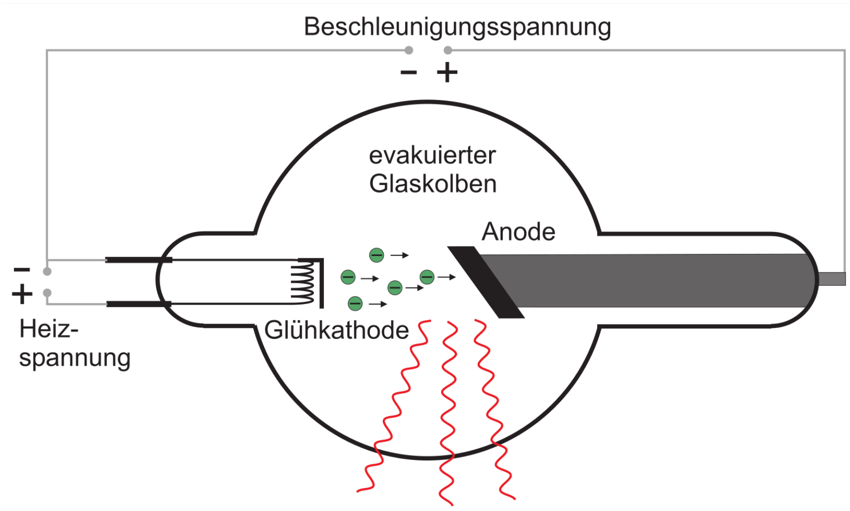


Figure 1.I:
Creating X-Ray. [Wag26b]

The maximum energy of an emitted photon is reached when an electron converts its entire kinetic energy into a single photon. This results in a minimum possible wavelength λ_{gr}, the cutoff wavelength (or short-wavelength limit), which is described by conservation of energy:

$$eU = h\nu_{gr} = \frac{hc}{\lambda_{gr}} \tag{1.2}$$

Here, h is Planck's quantum of action and c is the speed of light. In addition to the continuous spectrum, a characteristic line spectrum is generated if the voltage is sufficiently high. This occurs when electrons are knocked out of inner shells of the anode material and the resulting vacancies are filled by electrons from higher shells. For the transition from the n-th to the m-th shell, the energy $E_{n\to m}$ can be estimated according to Moseley's Law:

$$E_{n\to m} = hcR_\infty (Z-A)^2 \left(\frac{1}{m^2} - \frac{1}{n^2}\right) \tag{1.3}$$

Here, R_∞ denotes the Rydberg constant, Z the atomic number of the anode material (here molybdenum), and A a shielding constant, which is approximately $A \approx 1$ for K_α radiation ($n = 2 \to m = 1$).

1.2.2 Bragg Reflection

To measure the wavelengths of the generated X-ray radiation, crystals are used as diffraction gratings. Since the wavelengths of X-ray radiation are of the same order of magnitude as the interatomic distances in crystals, constructive interference occurs at the lattice planes of the crystal. This phenomenon is called Bragg reflection. An incident ray is reflected at various parallel lattice planes. For these reflected rays to overlap constructively, the path difference Δs must be an integer multiple of the wavelength λ. Bragg's Law describes this relationship:

$$2d\sin(\vartheta) = n\lambda, \quad n \in \mathbb{N} \tag{1.4}$$

Here, d is the distance between adjacent lattice planes, ϑ is the angle of incidence (glancing angle), and n is the order of reflection. By varying the angle ϑ and measuring the intensity, the X-ray spectrum can be recorded.

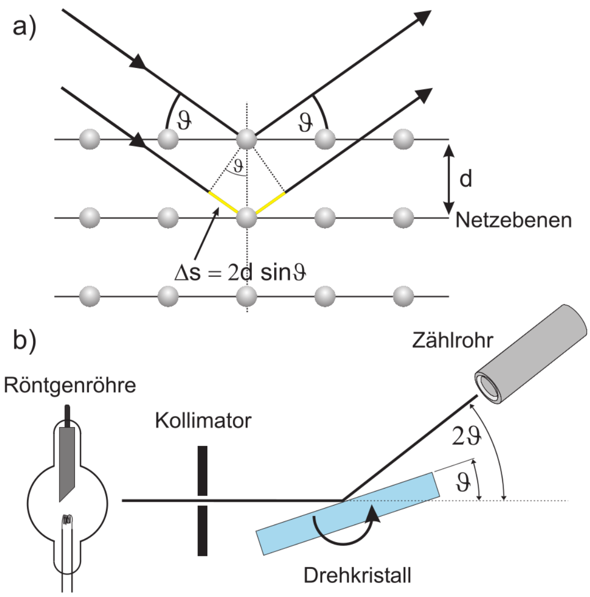


Figure 1.II:
Idea of Bragg scattering. [Wag26b]

1.2.3 Crystals and Lattice Structure

The LiF and NaCl crystals used in the experiment both possess a face-centered cubic structure. In this structure, a unit cell repeats periodically in space. There are four atoms (or ion pairs) per unit cell. For a cut along the principal planes, the lattice plane spacing d in terms of the lattice constant a is given by:

$$d = \frac{a}{2} \tag{1.5}$$

If the wavelength λ is known, the lattice constant a can be determined from the measured angle ϑ. Conversely, a known lattice constant enables the determination of microscopic quantities from macroscopically measurable properties. The Avogadro number N_A, which indicates the number of particles per mole, can be calculated via the volume of the unit cell. The volume of a unit cell is $V = (2d)^3$. With the molar mass M_{mol} and the density ρ of the crystal, it follows that:

$$N_A = \frac{4V_{mol}}{(2d)^3} = \frac{4M_{mol}}{\rho(2d)^3} = \frac{1}{2}\frac{M_{mol}}{\rho d^3} \tag{1.6}$$

This relationship allows the Avogadro number to be determined indirectly via X-ray diffraction. In the evaluation, the statistical nature of radioactive or count-rate-like events is also taken into account. Since the count rate M follows a Poisson process, the standard error of a single measurement is:

$$\sigma = \sqrt{M} \tag{1.7}$$

This value is used to incorporate the uncertainty of the measured intensities into the error propagation.

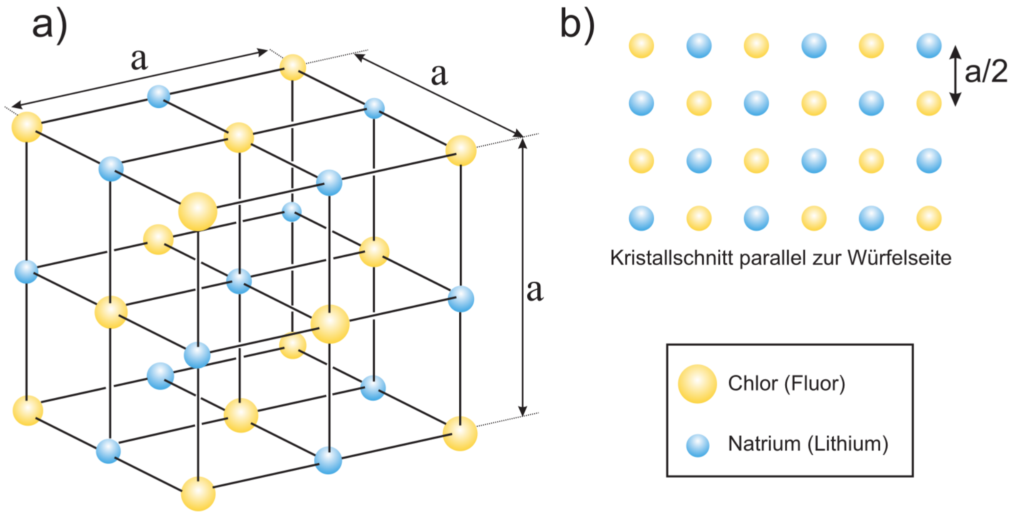


Figure 1.III:
Structure of the crystals in use; Here NaCl. [Wag26b]

Experiment 255: X-Ray

Annika Wüstle, Finn Zeumer

Date: 23.06.2026

Task 1)

Angle-Step: $\Delta\beta = 0.2^\circ$

Angle-Rang: $\beta \in [8^\circ, 22^\circ] \pm 0.05^\circ$

Acceleration Voltage: $U_B = 35$ kV ± 0.05 kV

Current: $I = 1$ mA ± 0.005

Gate-Time: $t = 5s \pm 0.5s$

Error estimations based on the accuracity of the scale/Display (shown digits)

Intervall for first order: $8^\circ - 11^\circ$

second order: $17.5^\circ - 21.5^\circ$

$\Big\}$ For Task 2

Task 2)

Angle-Step: $\Delta\beta = 0.1^\circ$

Angle-Rang: First order $\beta_1 \in [8^\circ, 11^\circ]$, Second Order $\beta_2 \in [17.5^\circ, 21.5^\circ]$

Acceleration Voltage: $U_B = 35$ kV

Current: $I = 1$ mA

Gate-Time: $t = 20s$

Task 3)

Angle-Step : $\Delta\beta = 0.1^\circ \equiv 0^\circ$, since we reset after every messurment

Angle-Rang : 7.5°

Acceleration Voltage: $U_B \in [20\,kV, 35\,kV]$, $\Delta U_B = 1.0\,kV$

Current : $I = 1\,mA$

Gate-Time : $t = 20s$

Tabel I) Counted x-Rays for a constant angle of 7.5°

Voltage $U_B \cdot 10^3$ [V]	Couts
20	0.90 ± 0.005
21	1.05
22	1.85
23	23.15
24	116.2 ± 0.05
25	321.6
26	405.1
27	573.1
28	673.8
29	769.2
30	865.0
31	945.8
32	1011.4
33	1090.8
34	1172.5
35	1235.7

Task 4)

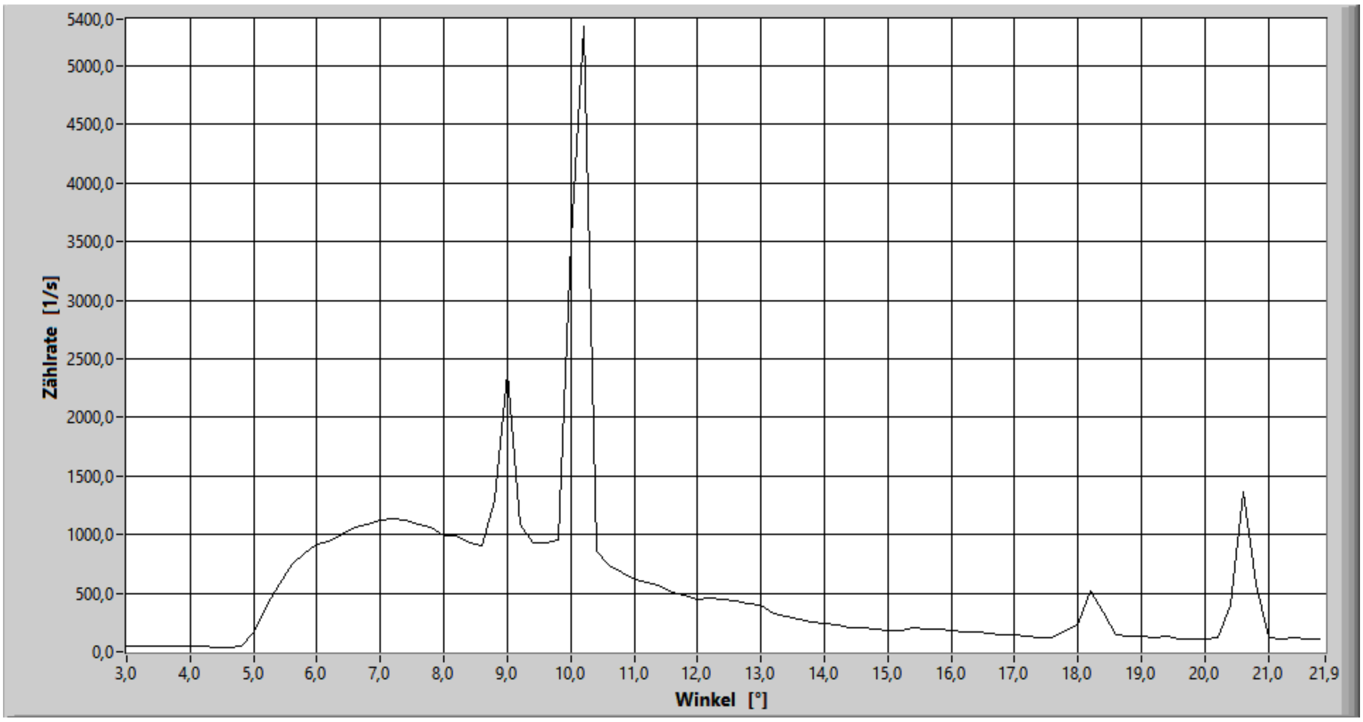
Same Settings as in Task 1. But $\beta \in [3^\circ, 18^\circ]$

Messprotokoll

→ Task 1

Di, 23. Jun 2026
14:54 Uhr

Röhrenspannung: 35,0 kV / Röhrenstrom: 1,0 mA / Scanbereich: 3,0° bis 21,9° / Schrittweite: 0,2° / Messzeit: 5,0 s



T1

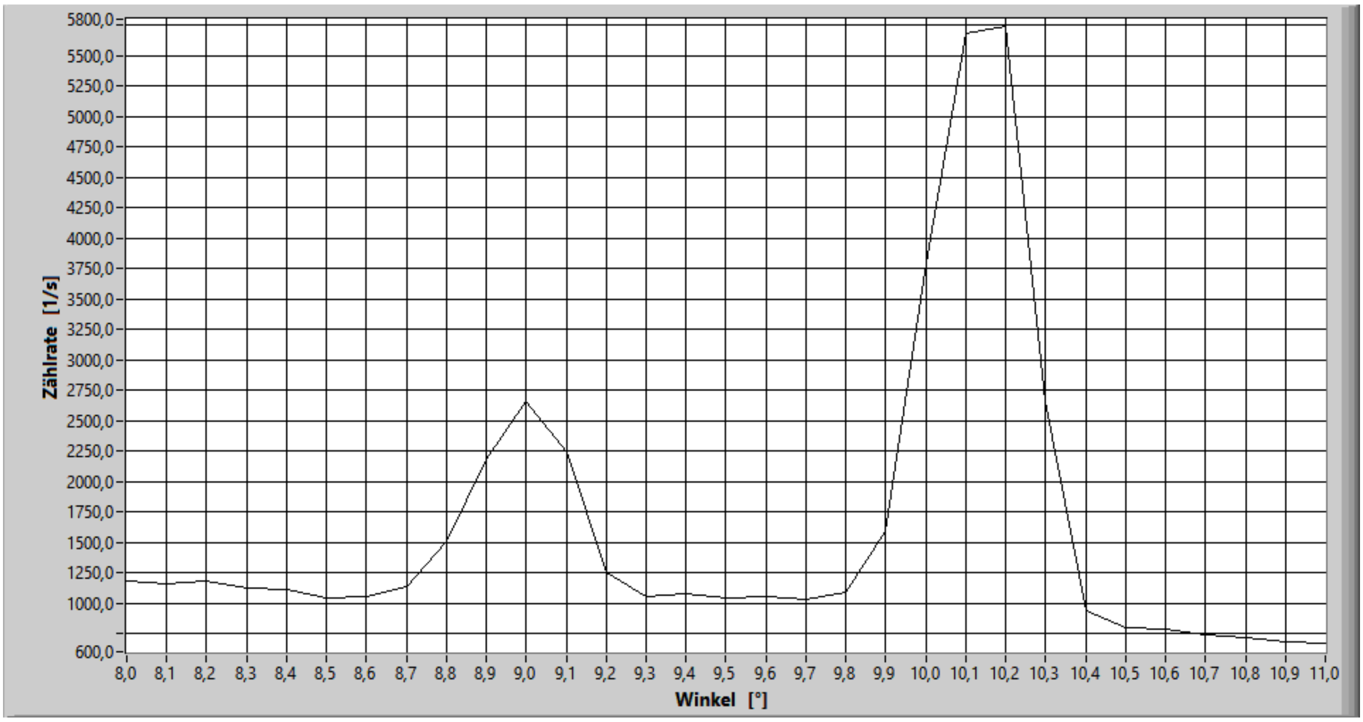
Winkel [°]	Zählrate [1/s]	Winkel [°]	Zählrate [1/s]	Winkel [°]	Zählrate [1/s]	Winkel [°]	Zählrate [1/s]	Winkel [°]	Zählrate [1/s]
3,0	51,6	7,0	1125,0	11,0	621,2	15,0	186,8	19,0	137,2
3,2	54,0	7,2	1135,6	11,2	596,2	15,2	186,4	19,2	121,6
3,4	49,6	7,4	1123,6	11,4	564,0	15,4	200,0	19,4	127,2
3,6	54,0	7,6	1084,0	11,6	512,0	15,6	191,8	19,6	106,8
3,8	47,0	7,8	1062,8	11,8	477,4	15,8	190,4	19,8	114,0
4,0	49,0	8,0	995,6	12,0	446,6	16,0	180,0	20,0	110,4
4,2	46,4	8,2	992,0	12,2	456,8	16,2	168,8	20,2	126,6
4,4	41,0	8,4	931,4	12,4	446,0	16,4	166,4	20,4	400,4
4,6	40,8	8,6	911,0	12,6	433,8	16,6	152,2	20,6	1368,0
4,8	53,6	8,8	1291,8	12,8	410,6	16,8	147,4	20,8	574,6
5,0	171,0	9,0	2383,6	13,0	398,8	17,0	146,6	21,0	119,6
5,2	400,4	9,2	1081,4	13,2	328,8	17,2	133,4	21,2	104,0
5,4	577,8	9,4	934,4	13,4	302,4	17,4	124,4	21,4	115,4
5,6	746,6	9,6	935,4	13,6	277,6	17,6	126,8	21,6	104,6
5,8	828,6	9,8	953,8	13,8	258,6	17,8	179,2	21,8	103,6
6,0	916,2	10,0	3509,8	14,0	242,4	18,0	225,4	0,0	0,0
6,2	939,8	10,2	5340,2	14,2	230,2	18,2	517,2	0,0	0,0
6,4	1004,8	10,4	854,6	14,4	199,8	18,4	340,4	0,0	0,0
6,6	1062,6	10,6	740,4	14,6	202,2	18,6	145,4	0,0	0,0
6,8	1092,8	10,8	681,0	14,8	194,2	18,8	137,4	0,0	0,0

Messprotokoll

Task 2 → 1. Order

Di, 23. Jun 2026
15:08 Uhr

Röhrenspannung: 35,0 kV / Röhrenstrom: 1,0 mA / Scanbereich: 8,0° bis 11,0° / Schrittweite: 0,1° / Messzeit: 20,0 s



T2 — 1.Ord.

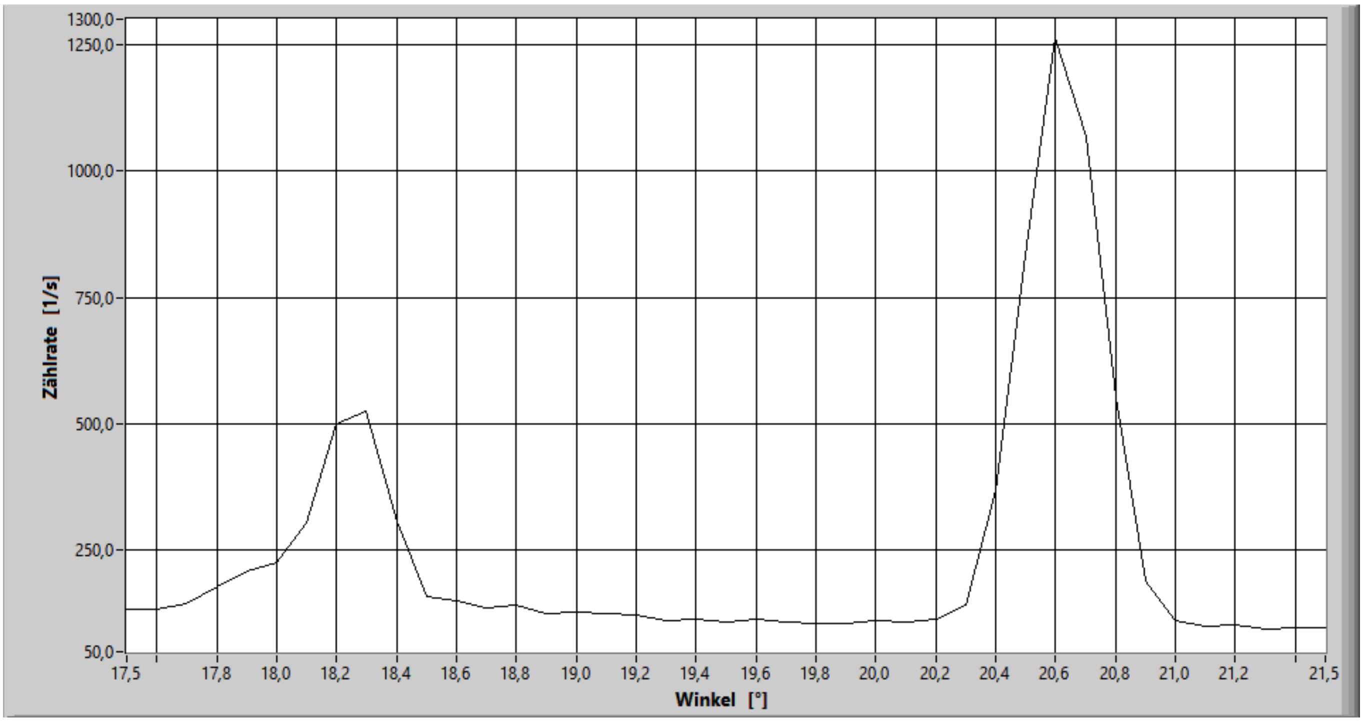
Winkel [°]	Zählrate [1/s]	Winkel [°]	Zählrate [1/s]
8,0	1183,8	10,0	3800,2
8,1	1153,8	10,1	5679,6
8,2	1182,7	10,2	5739,6
8,3	1120,2	10,3	2653,4
8,4	1114,5	10,4	932,3
8,5	1038,0	10,5	795,7
8,6	1050,0	10,6	787,0
8,7	1136,8	10,7	741,4
8,8	1503,0	10,8	714,1
8,9	2183,8	10,9	676,4
9,0	2664,8	11,0	667,2
9,1	2243,6	0,0	0,0
9,2	1247,3	0,0	0,0
9,3	1051,3	0,0	0,0
9,4	1073,6	0,0	0,0
9,5	1042,8	0,0	0,0
9,6	1053,2	0,0	0,0
9,7	1030,3	0,0	0,0
9,8	1083,6	0,0	0,0
9,9	1590,7	0,0	0,0

Messprotokoll

Task 2 – 2. Order

Di, 23. Jun 2026
15:27 Uhr

Röhrenspannung: 35,0 kV / Röhrenstrom: 1,0 mA / Scanbereich: 17,5° bis 21,5° / Schrittweite: 0,1° / Messzeit: 20,0 s



T2 – 2. Ord.

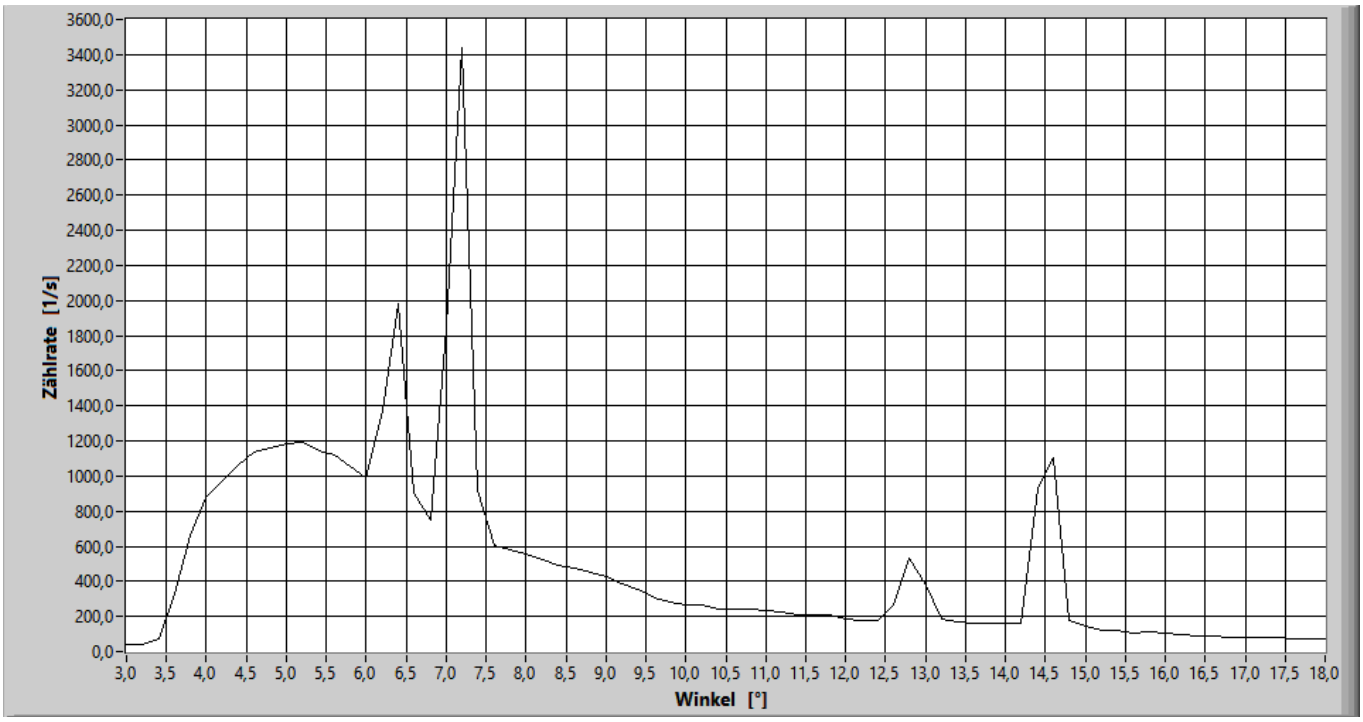
Winkel [°]	Zählrate [1/s]	Winkel [°]	Zählrate [1/s]	Winkel [°]	Zählrate [1/s]
17,5	133,8	19,5	109,7	21,5	96,8
17,6	132,8	19,6	113,0	0,0	0,0
17,7	143,8	19,7	108,8	0,0	0,0
17,8	177,8	19,8	106,8	0,0	0,0
17,9	208,1	19,9	107,0	0,0	0,0
18,0	225,3	20,0	111,3	0,0	0,0
18,1	304,9	20,1	108,7	0,0	0,0
18,2	501,2	20,2	113,9	0,0	0,0
18,3	524,8	20,3	141,8	0,0	0,0
18,4	306,7	20,4	368,5	0,0	0,0
18,5	158,0	20,5	832,2	0,0	0,0
18,6	150,0	20,6	1262,6	0,0	0,0
18,7	135,7	20,7	1068,7	0,0	0,0
18,8	142,2	20,8	554,6	0,0	0,0
18,9	125,0	20,9	190,7	0,0	0,0
19,0	128,3	21,0	112,5	0,0	0,0
19,1	125,7	21,1	101,3	0,0	0,0
19,2	122,3	21,2	102,8	0,0	0,0
19,3	112,8	21,3	94,6	0,0	0,0
19,4	115,3	21,4	98,6	0,0	0,0

Messprotokoll

Task 4

Di, 23. Jun 2026
15:50 Uhr

Röhrenspannung: 35,0 kV / Röhrenstrom: 1,0 mA / Scanbereich: 3,0° bis 18,0° / Schrittweite: 0,2° / Messzeit: 5,0 s



T4

Winkel [°]	Zählrate [1/s]	Winkel [°]	Zählrate [1/s]	Winkel [°]	Zählrate [1/s]	Winkel [°]	Zählrate [1/s]
3,0	41,2	7,0	1840,6	11,0	230,4	15,0	143,6
3,2	38,6	7,2	3439,6	11,2	222,4	15,2	123,8
3,4	71,4	7,4	915,8	11,4	209,8	15,4	122,4
3,6	328,6	7,6	605,8	11,6	206,4	15,6	107,0
3,8	661,6	7,8	579,4	11,8	208,0	15,8	112,2
4,0	876,8	8,0	555,4	12,0	185,2	16,0	104,8
4,2	976,8	8,2	524,2	12,2	179,6	16,2	95,8
4,4	1060,4	8,4	493,8	12,4	173,8	16,4	89,6
4,6	1139,2	8,6	471,4	12,6	263,4	16,6	89,6
4,8	1161,4	8,8	452,2	12,8	530,8	16,8	79,8
5,0	1182,0	9,0	426,0	13,0	378,6	17,0	79,2
5,2	1195,8	9,2	386,2	13,2	186,2	17,2	78,0
5,4	1140,6	9,4	350,4	13,4	165,6	17,4	80,0
5,6	1116,2	9,6	303,2	13,6	163,8	17,6	76,2
5,8	1052,0	9,8	279,6	13,8	162,8	17,8	71,2
6,0	994,2	10,0	269,0	14,0	157,6	18,0	69,8
6,2	1370,0	10,2	262,4	14,2	161,8	0,0	0,0
6,4	1984,8	10,4	242,4	14,4	928,6	0,0	0,0
6,6	898,0	10,6	238,8	14,6	1105,0	0,0	0,0
6,8	747,0	10,8	241,2	14,8	174,6	0,0	0,0

3. Experimental Procedure

3.1 Measurement Method

The experiment is based on the rotating crystal method for spectrometry. In this approach, a crystal holder equipped with the respective detection crystal (initially LiF, later NaCl) is positioned in the X-ray beam path. A Geiger-Müller counter tube serves as the detector for intensity-measured X-ray radiation. The actual measurement process is performed automatically by a control program on the computer. First, a safety check of the device is carried out, ensuring that the lead-glass window is closed before the high voltage is activated.

For recording the bremsstrahlung spectrum, the crystal is rotated over a specific angular range while, for each angular position, the count rate of detected photons is measured over a predetermined time period. To increase accuracy in determining the characteristic lines (K_α and K_β), after a coarse preliminary scan, detailed measurements are performed only in the regions of expected maxima. Here, the step size is reduced to capture the shape of the peaks precisely.

Another part of the experiment investigates the relationship between the count rate and the accelerating voltage of the X-ray tube at a constant detection angle. Here, the voltage is varied stepwise while the count rate is registered for each setting. Finally, the entire measurement sequence is repeated with an NaCl crystal to determine the lattice constant using the already determined wavelengths, and from this, the Avogadro number. All measured values are recorded digitally and documented directly.

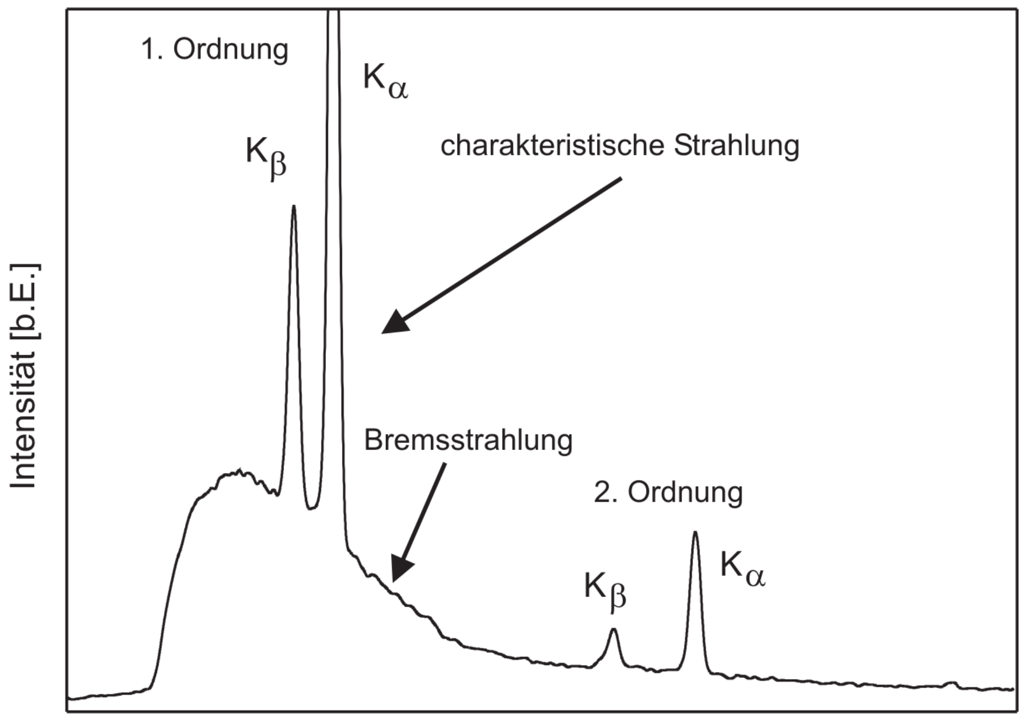


Figure 3.I:
Example of a X-Ray spectrum. [Wag26b]

3.2 Experimental Setup

The experimental setup consists of three main components: the X-ray source, the goniometric detection system, and the evaluation electronics. The X-ray tube features a water-cooled copper holder and a molybdenum anode, arranged inside the vacuum glass bulb opposite a heated tungsten cathode. A high-voltage source generates the electric field for accelerating the electrons.

The detection system includes a precision goniometer, on which the crystal holder and the counter tube are mounted movably. The rotation of the crystal and the simultaneous movement of the counter occur synchronously according to the Bragg angle (1:2 relation between crystal and counter rotation), so that the detector always captures the reflected beam. The counter tube is connected to an amplifier and a pulse counter, which digitizes the signals and forwards them to a computer. For safety, the entire beam path is embedded in a lead housing, accessible through a swivel lead-glass window. An internal safety switch ensures that the high voltage is switched off immediately as soon as the window is opened.

Sketch of the Experimental Setup

The image shows the arrangement of X-ray tube, collimator, the rotatable crystal holder, the Geiger-Müller counter tube, and the connections to the control unit.

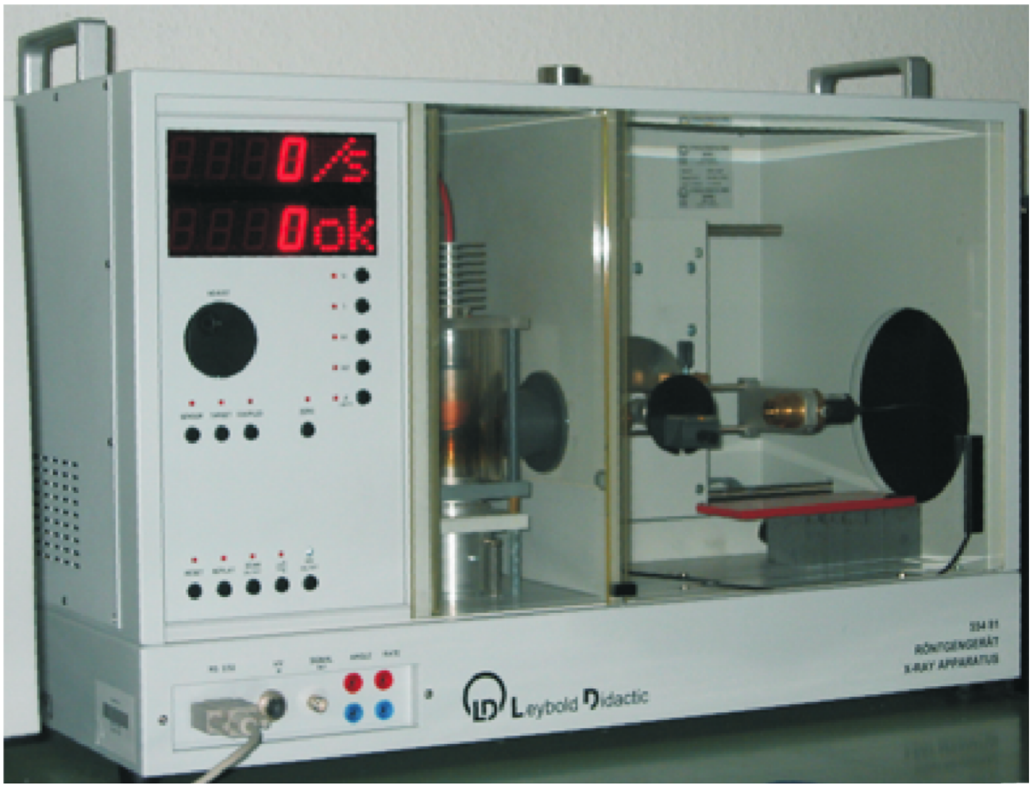


Figure 3.II:
Schematic setup of the X-ray spectrometer following the rotating crystal method.

4. Evaluation

Uncertanty Calculation

For the statistical evaluation of n messurments x_i, the follwoing equations are defined: [Wag25]:

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i \qquad \text{Arithmetic Mean} \tag{4.1}$$

$$\sigma^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2 \qquad \text{Variance} \tag{4.2}$$

$$\sigma = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2} \qquad \text{Standard Deviation} \tag{4.3}$$

$$\Delta\bar{x} = \frac{\sigma}{\sqrt{n}} = \sqrt{\frac{1}{n(n-1)}\sum_{i=1}^{n}(\bar{x} - x_i)^2} \qquad \text{Uncertainty of the Mean} \tag{4.4}$$

$$\Delta f = \sqrt{\left(\frac{\partial f}{\partial x}\Delta x\right)^2 + \left(\frac{\partial f}{\partial y}\Delta y\right)^2} \qquad \text{Gaussian Error Propagation for } f(x,y) \tag{4.5}$$

$$\Delta f = \sqrt{(\Delta x)^2 + (\Delta y)^2} \qquad \text{Uncertainty for } f = x + y \tag{4.6}$$

$$\Delta f = |a|\Delta x \qquad \text{Uncertainty for } f = ax \tag{4.7}$$

$$\frac{\Delta f}{|f|} = \sqrt{\left(\frac{\Delta x}{x}\right)^2 + \left(\frac{\Delta y}{y}\right)^2} \qquad \text{Relative Uncertainty for } f = xy \text{ or } f = x/y \tag{4.8}$$

$$\sigma = \frac{|a_{lit} - a_{gem}|}{\sqrt{\Delta a_{lit}^2 + \Delta a_{gem}^2}} \qquad \text{Significant Deviation Calculation} \tag{4.9}$$

4.1 X-Ray Spectrum with Lithium Fluoride

In the first part, the X-ray spectrum obtained with the mounted Lithium Fluoride (LiF) crystal is evaluated. This spectrum is used to estimate Planck's constant h, determine the wavelengths corresponding to the peak positions – the characteristic lines – and investigate the influence of the acceleration voltage on the estimated Planck's constant.

4.1.1 Calculating Planck's Constant

In this section, we use the measurement data and apply a regression to the nearly linear portion at the beginning. For the angle, we use an uncertainty of $\Delta\vartheta = 0.05°$, and for the counts, we use Equation 4.3. The linear regression $f(\vartheta) = m \cdot \vartheta + b$ has a slope m and crosses the y-axis at b. Both values can be found in Plot 4.I.

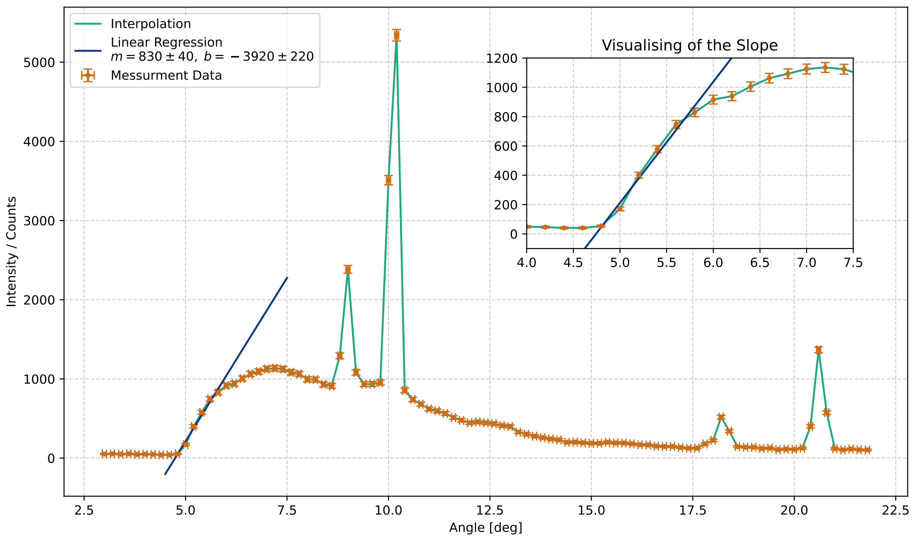


Figure 4.I:
Measurement data interpolated alongside the linear regression.

Setting the linear regression equal to zero and solving for ϑ, we obtain the critical angle:

$$\vartheta_{\text{crit}} = -\frac{b}{a}, \quad \Delta\vartheta_{\text{crit}} = \vartheta_{\text{crit}} \cdot \sqrt{\frac{\Delta m^2}{m^2} + \frac{\Delta b^2}{b^2}} \tag{4.10}$$

Thus, the critical angle can be expressed as

$$\underline{\vartheta_{\text{crit}} = (4.7 \pm 0.3)\,°} \tag{4.11}$$

Using Equation 1.4 and rearranging it to get

$$\lambda_{\text{crit}} = \frac{2d}{n} \sin(\vartheta)\,, \quad \Delta\lambda_{\text{crit}} = \lambda_{\text{crit}} \cdot \sqrt{\frac{\Delta\vartheta^2}{\tan^2(\vartheta)}}\,. \tag{4.12}$$

Using the values $n = 1$ and $d = 201.4\,\text{pm}$ as well as the calculated angle ϑ_{crit}, the wavelength is calculated to be

$$\underline{\lambda_{\text{crit}} = (3.30 \pm 0.21) \cdot 10^{-11}\,\text{m}} \tag{4.13}$$

With the critical wavelength determined, we use Equation 1.2 and rearrange it to calculate Planck's constant using the formula

$$h = \frac{U_B \lambda_{\text{crit}}}{c} e\,, \quad \Delta h = h \cdot \sqrt{\frac{\Delta\lambda_{\text{crit}}^2}{\lambda_{\text{crit}}^2} + \frac{\Delta U_B{}^2}{U_B{}^2}} \tag{4.14}$$

Given the constants e and c from the script [Wag26b], the calculated λ_{crit}, and $U_B = (35.00 \pm 0.05)\,\text{kV}$ set manually, Planck's constant can be calculated to

$$\boxed{h = (6.2 \pm 0.4) \cdot 10^{-34}\,\text{Js}} \tag{4.15}$$

This value differs from the accepted literature value $h = 6.62607015 \cdot 10^{-34}\,\text{J s}$ by 1.07σ and is therefore statistically consistent. Furthermore, it is possible to estimate the onset of the second order. Again, Equation 1.4 is used. The angle for the second order can be calculated by

$$\vartheta_{\text{2ord}} = \operatorname{asin}\left(\frac{\lambda_{\text{crit}}}{d}\right), \quad \Delta\vartheta_{\text{2ord}} = \vartheta_{\text{2ord}} \cdot \sqrt{-\frac{\Delta\lambda_{\text{crit}}^2}{(\lambda_{\text{crit}}^2 - d^2)\operatorname{asin}^2\left(\frac{\lambda_{\text{crit}}}{d}\right)}} \tag{4.16}$$

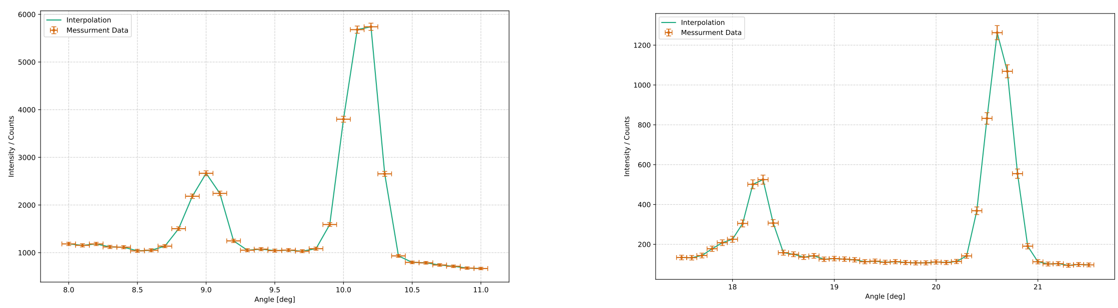
Substituting the known values into the equation, the angle is calculated to be

$$\boxed{\vartheta_{\text{2ord}} = (9.4 \pm 0.6)^\circ} \tag{4.17}$$

meaning the second-order K-lines will begin approximately $(9.4 \pm 0.6)^\circ$ beyond where the first order starts.

4.1.2 Evaluation of the K-Lines

Next, we focus on the K-lines themselves in order to identify the K_α- and K_β-lines for Mo. Both orders were measured with a higher resolution in order to improve the results. Figure 4.II shows the spectra being analyzed in this section.



(a) **(b)**

Figure 4.II:
(a) High-resolution spectrum of the first-order and (b) second-order K-lines.

All four peaks are fitted using a Gaussian regression, allowing the angle of each peak to be determined quite precisely. In Figure 4.III, the original data from 4.IIa is shown alongside the Gaussian regression for the first-order peaks. The parameters for the Gaussian regressions are also displayed in the plot – they can be found in the legend under each regression curve. Here, μ corresponds to the angle of the peak. By substituting the angles obtained from the Gaussian regression into Equation 4.12, the wavelength for each K-line can be calculated. The results are presented in Table 4.I.

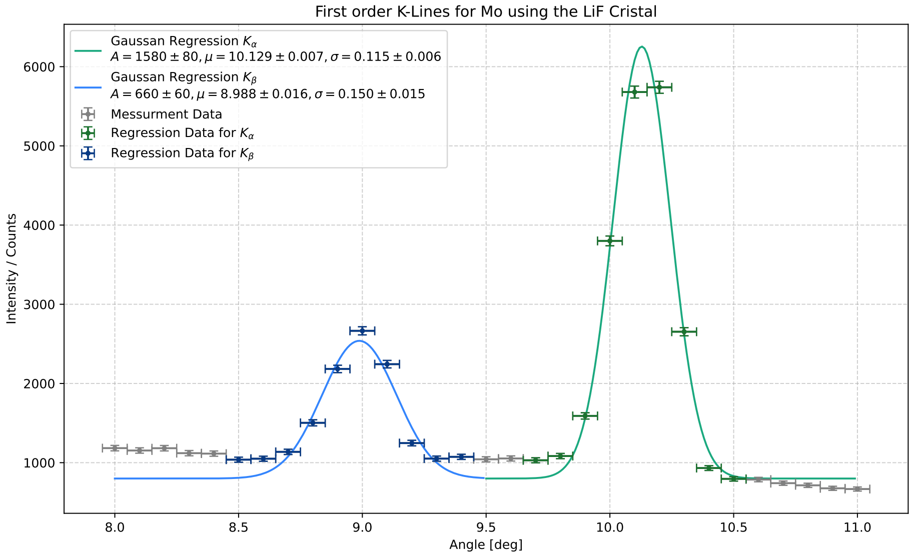


Figure 4.III:
First-order X-ray spectrum of Mo using a LiF crystal.

Furthermore, using the Gaussian regression, the half-width (FWHM[1]) can be calculated using the formula

$$\text{FWHM} = 2\sqrt{2\log(2)}\sigma\,, \quad \Delta\text{FWHM} = \text{FWHM} \cdot \sqrt{\frac{\Delta\sigma^2}{\sigma^2}}\,. \tag{4.18}$$

These results are also listed in Table 4.I. For the second-order peaks, the data from Figure 4.IIb is used. The Gaussian regression as well as the parameters can be found in Figure 4.IV. Analogous to the calculations for the first-order K-lines, the angles of the second-order K-lines are evaluated. Those result can also be found in the Table 4.I.

Table 4.I:
Results for the second task. The literature values $\lambda_\alpha = 71.1$ and $\lambda_\beta = 63.1$ were used. [Wag26b]

K-Line	angle [deg]	λ_{Mo} [pm]	Standard Deviation σ	FWHM [deg]	FWHM [pm]
α_1	10.129 ± 0.007	71.3 ± 0.3	0.66	0.272 ± 0.014	1.91 ± 0.10
α_2	20.6214 ± 0.0017	70.86 ± 0.16	1.50	0.298 ± 0.003	2.49 ± 0.24
α_{avg}		71.1 ± 0.4	0.00	0.285 ± 0.014	2.20 ± 0.26
β_1	8.988 ± 0.016	63.0 ± 0.3	0.33	0.35 ± 0.03	1.048 ± 0.012
β_2	18.230 ± 0.021	63.24 ± 0.17	0.82	0.39 ± 0.05	1.38 ± 0.17
β_{avg}		63.1 ± 0.4	0.00	0.37 ± 0.06	1.21 ± 0.17

None of the standard deviations for any wavelength deviate significantly from the literature values. Although not shown in the table, none of the standard deviations for teh FWHM values (pm) between two K-lines are significant either. The standard deviation between α_1 and α_2 is 2.23σ, and for the β K-lines it is 1.95σ. Statistically, it can be assumed that the first- and second-order K-lines have the same half-width.

[1]"FWHM" stands for "Full Width at Half Maximum"

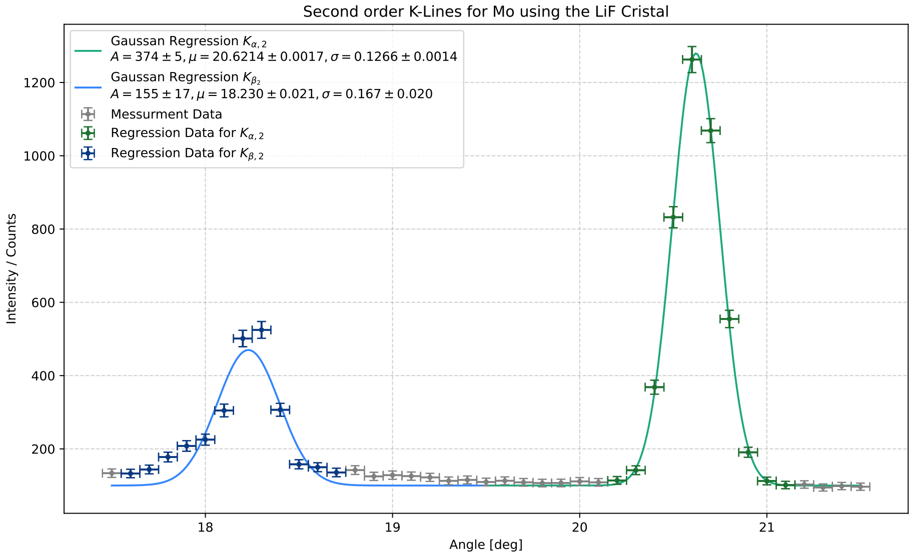


Figure 4.IV:
Second-order X-ray spectrum of Mo using a LiF crystal.

4.1.3 Counts as a Function of Acceleration Voltage

In this section, the impact of the acceleration voltage U_B is evaluated. To this end, the counts per second are plotted against the voltage U_B, as shown in Figure 4.V. A linear regression was applied to the nearly linear region; the regression line as well as its fit parameters can be found in the plot.

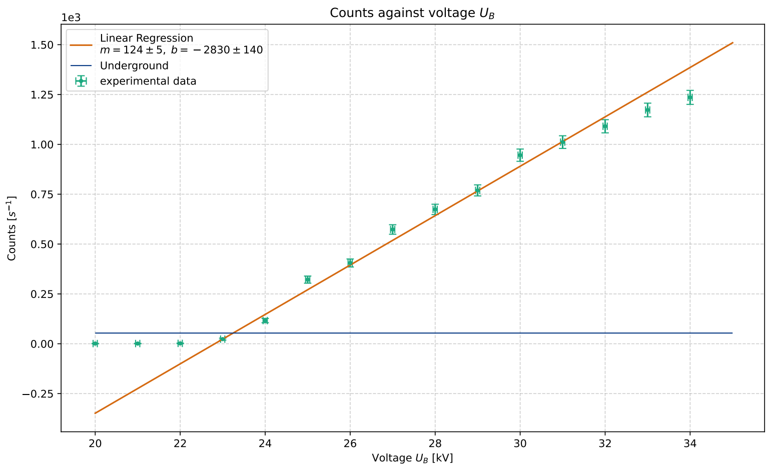


Figure 4.V:
Linear regression through the increasing part of the data points.

By determining the intersection of the background rate $n_U = (54 \pm 7)\mathrm{s}^{-1}$ with the linear regression, the operating voltage U_E can be determined. The uncertainty is calculated by propagating the uncertainty of the background rate and recomputing the intersection accordingly. Finally, the operating voltage is determined to be

$$\underline{U_E = (2.325 \pm 0.006) \cdot 10^4\ \mathrm{V}} \tag{4.19}$$

Using Equation 4.14, but replacing U_B with the calculated U_E, Planck's constant can be calculated to be

$$\boxed{h = (6.53 \pm 0.05) \cdot 10^{-34}\,\mathrm{J\,s}} \tag{4.20}$$

The standard deviation from the literature value is 2.02σ and is thus not statistically significant.

4.2 X-Ray Spectrum with Sodium Chloride

Next, the crystal is replaced with a Sodium Chloride (NaCl) crystal. The raw signal is shown in Figure 4.VI. As before, the plot displays four peaks corresponding to two orders each: the K_α and K_β K-lines are again clearly visible. In this section, two primary objectives are accomplished through the utilization of these measurements: first, determination of the lattice constant g for NaCl using Bragg's law, and second, calculation of Avogadro's number based on the known density and molar mass of sodium chloride. By analyzing the diffraction angles obtained from the measured K-line positions and applying the same analytical framework established for the LiF crystal, structural parameters characteristic of the NaCl crystal structure can be extracted.

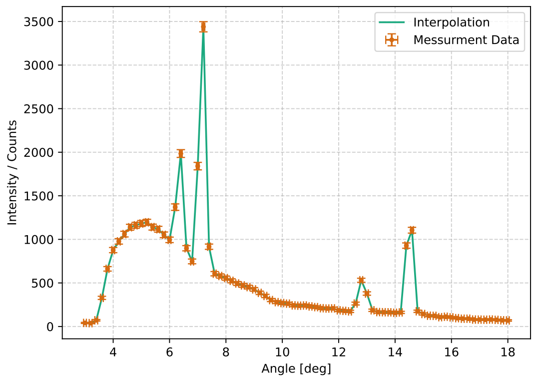


Figure 4.VI:
Raw X-Ray spectrum for NaCl.

4.2.1 Lattice of the NaCl Cristal

To analyze this signal properly, a Gaussian regression is used once more, allowing the angle of each peak to be determined. Figure 4.VII shows each Gaussian regression along with the corresponding fit parameters. The angles as well as further results can be found in Table 4.II. The angles correspond to the μ-values with an uncertainty of $\sqrt{\sigma}$. The expected wavelengths are included to verify that the general magnitude of the angles is consistent with the previous results. These were calculated using the literature value for the lattice constant g. Determining the lattice constant from the measurement data can be done using the equation

$$g = \frac{\lambda n}{\sin(\vartheta)}\,, \qquad \Delta g = g \cdot \sqrt{\frac{\Delta\vartheta^2}{\tan^2(\vartheta)} + \frac{\Delta\lambda^2}{\lambda^2}}\,. \tag{4.21}$$

Here, λ denotes the wavelength of Mo calculated in Section 4.1, namely $\lambda_\alpha = (71.1 \pm 0.4)\,\mathrm{pm}$ and $\lambda_\beta = (63.1 \pm 0.4)\,\mathrm{pm}$. The angles ϑ are those listed in the table. Each individual lattice constant is also given in the table. The mean is calculated using Equation 4.1.

Table 4.II:
X-ray diffraction analysis results. For the expected wavelength, the literature value $g_{\mathrm{NaCl}} = 564\,\mathrm{pm}$ was used. Source: https://de.wikipedia.org/wiki/Natriumchlorid

K-Line	angle ϑ [deg]	Expected λ_{Mo} [pm]	Lattice constant NaCl [pm]	Standard Deviation σ
α_1	7.17 ± 0.16	70.3 ± 1.5	570 ± 13	0.47
β_1	6.3 ± 0.3	62 ± 3	640 ± 30	2.33
α_2	14.51 ± 0.15	70.7 ± 0.7	567 ± 7	0.54
β_2	12.8 ± 0.4	62.6 ± 2.0	641 ± 20	3.78
Mean			606 ± 19	2.25

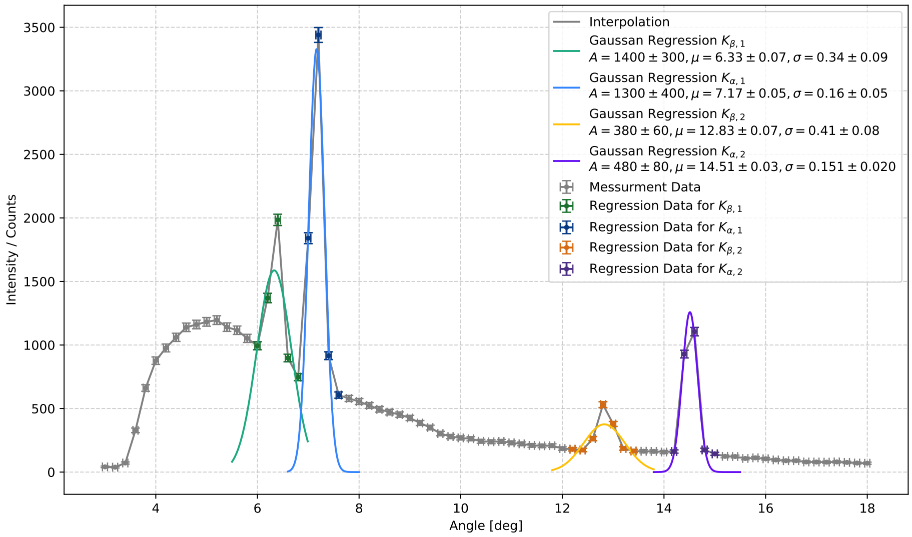


Figure 4.VII:
Sodium chloride spectrum with a Gaussian regression through each K-line.

The standard deviation is calculated according to Equation 4.9. Only the deviation for β_2 is significant. In general, the β-lines exhibit a trend of larger deviations. The results will be discussed in Chapter 6..

4.2.2 Evaluation of the Avogadro Number

Finally, the lattice constant can be used for the determination of the Avogadro number N_A. Equation 1.6 is employed:

$$N_A = \frac{4M_{mol}}{\rho g^3}\,, \quad \Delta N_A = 3N_A \cdot \sqrt{\frac{\Delta g^2}{g^2}}\,. \tag{4.22}$$

Using the mean for the lattice constant, as well as $\rho = 2.146\,\mathrm{g/cm^3}$ and $M = 58.44\,\mathrm{g}$, yields the Avogadro number:

$$\boxed{N_A = (4.9 \pm 0.4) \cdot 10^{23}\,\frac{1}{\mathrm{mol}}} \tag{4.23}$$

The literature value is $N_A = 6.02214076 \cdot 10^{23}$; hence, the standard deviation is 2.60σ and thus not statistically significant. Since the values for the K_α-lines showed higher accuracy, the Avogadro number is also calculated for α_1 as a reference:

$$\boxed{N_{A_\alpha} = (5.8 \pm 0.4) \cdot 10^{23}\,\frac{1}{\mathrm{mol}}} \tag{4.24}$$

Here, the standard deviation is only 0.48σ, which is likewise not statistically significant but closer to the literature value.

5. Discussion

5.1 Summary

In this experiment, several key quantities were determined and evaluated. In the first part, Planck's constant was estimated using the critical angle obtained from the X-ray spectrum of the LiF crystal. The result

$$\boxed{h = (6.2 \pm 0.4) \cdot 10^{-34}\,\mathrm{Js}} \tag{5.1}$$

yields a standard deviation of 1.07σ from the literature value and is therefore not statistically significant. Additionally, the onset of the second diffraction order was estimated to occur at

$$\boxed{\vartheta_{2\mathrm{ord}} = (9.4 \pm 0.6)\,^\circ} \tag{5.2}$$

beyond the starting point of the first order. The wavelengths of the molybdenum K_α- and K_β-lines were determined to be

$$\boxed{K_\alpha = (7.11 \pm 0.04) \cdot 10^{-11}\,\mathrm{m}} \tag{5.3}$$

$$\boxed{K_\beta = (6.31 \pm 0.04) \cdot 10^{-11}\,\mathrm{m}} \tag{5.4}$$

with a standard deviation of 0.00σ for both lines, indicating excellent agreement with the literature values. Furthermore, the full width at half maximum (FWHM) was calculated for the K_α-line, yielding

$$\boxed{\alpha_1 = (1.91 \pm 0.10) \cdot 10^{-12}\,\mathrm{m}} \tag{5.5}$$

for the first-order α_1-peak. The results for all FWHM values are summarized in Table 4.I. None of the standard deviations are significant, as discussed in more detail in Section 6.2. In addition, Planck's constant was independently estimated using the operating voltage U_E instead of the acceleration voltage, resulting in

$$\boxed{h = (6.53 \pm 0.05) \cdot 10^{-34}\,\mathrm{J\,s}} \tag{5.6}$$

with a standard deviation of 2.02σ, which is likewise not statistically significant. Upon replacing the crystal with NaCl, the lattice constant was determined to be

$$\boxed{g = (6.06 \pm 0.19) \cdot 10^{-10}\,\mathrm{m}} \tag{5.7}$$

with a standard deviation of 2.25σ. Based on this lattice constant, Avogadro's number was calculated as

$$\boxed{N_A = (4.9 \pm 0.4) \cdot 10^{23}\,\frac{1}{\mathrm{mol}}} \tag{5.8}$$

5.2 Analysis of the Data

Several observations can be made regarding the quality and consistency of the obtained results. Regarding the FWHM, it is noteworthy that the values for α_1 and α_2 as well as the β_1 and β_2 are statistically compatible with one another, suggesting that the first- and second-order measurements yield comparable peak widths. A notable discrepancy is observed in the lattice constant results for NaCl. The β-lines consistently yield lower-quality results than the α-lines. All values overestimate the literature value for the lattice constant, and the β-lines exhibit both lower accuracy and lower precision, as reflected in their higher uncertainties. This behavior was anticipated when examining the expected wavelengths, since all of them are smaller than the literature values for Mo, indicating a systematic offset that particularly affects the β-line measurements. A similar trend is observed in the calculation of Avogadro's number. When using the mean lattice constant – which includes all four K-lines – the standard deviation amounts to 2.60σ. However, when the calculation is restricted to the α_1-line, the standard deviation drops to only 0.48σ:

$$\boxed{N_{A\alpha} = (5.8 \pm 0.4) \cdot 10^{23}\,\frac{1}{\mathrm{mol}}} \tag{5.9}$$

This substantial improvement suggests the presence of a systematic problem that disproportionately affects the β-line measurements, likely linked to the aforementioned discrepancies in the expected wavelengths.

One reason might be that the reselution of the sepctrum for the NaCl was quite low, increasing the Gate-Time might already fix the systematic problem.

5.3 Criticism

Some of the uncertainties obtained in this experiment are notably small, implying a high degree of precision. However, this apparent precision is likely unrealistic, as several error sources were neglected in the analysis. In particular, the actual measurement precision of the apparatus itself was not taken into account. Instead, the uncertainty was estimated solely on the basis of the number of digits displayed on the instrument, with the simplifying assumption that this corresponds to the readout error divided by two as well as statistical errors. Additional contributing factors, such as mechanical positioning accuracy of the goniometer, temperature drift, and counting statistics at low count rates, were omitted entirely and may lead to an underestimation of the true uncertainties.

However, this would not affect the accuracy of the results, hence all calculated values where already statistically relevant and reflect[1] the theory quite well, and would still do so after a more complex uncertainty calculation.

Still, an improvment of the messurment apparatus would increase the accuracy as well. And so would more messurments do as well.

[1] pan intended

6. Discussion

6.1 Summary

In this experiment, several key quantities were determined and evaluated. In the first part, Planck's constant was estimated using the critical angle obtained from the X-ray spectrum of the LiF crystal. The result

$$\boxed{h = (6.2 \pm 0.4) \cdot 10^{-34}\,\mathrm{Js}} \tag{6.1}$$

yields a standard deviation of 1.07σ from the literature value and is therefore not statistically significant. Additionally, the onset of the second diffraction order was estimated to occur at

$$\boxed{\vartheta_{\mathrm{2ord}} = (9.4 \pm 0.6)\,^{\circ}} \tag{6.2}$$

beyond the starting point of the first order. The wavelengths of the molybdenum K_α- and K_β-lines were determined to be

$$\boxed{K_\alpha = (7.11 \pm 0.04) \cdot 10^{-11}\,\mathrm{m}} \tag{6.3}$$

$$\boxed{K_\beta = (6.31 \pm 0.04) \cdot 10^{-11}\,\mathrm{m}} \tag{6.4}$$

with a standard deviation of 0.00σ for both lines, indicating excellent agreement with the literature values. Furthermore, the full width at half maximum (FWHM) was calculated for the K_α-line, yielding

$$\boxed{\alpha_1 = (1.91 \pm 0.10) \cdot 10^{-12}\,\mathrm{m}} \tag{6.5}$$

for the first-order α_1-peak. The results for all FWHM values are summarized in Table 4.I. None of the standard deviations are significant, as discussed in more detail in Section 6.2. In addition, Planck's constant was independently estimated using the operating voltage U_E instead of the acceleration voltage, resulting in

$$\boxed{h = (6.53 \pm 0.05) \cdot 10^{-34}\,\mathrm{J\,s}} \tag{6.6}$$

with a standard deviation of 2.02σ, which is likewise not statistically significant. Upon replacing the crystal with NaCl, the lattice constant was determined to be

$$\boxed{g = (6.06 \pm 0.19) \cdot 10^{-10}\,\mathrm{m}} \tag{6.7}$$

with a standard deviation of 2.25σ. Based on this lattice constant, Avogadro's number was calculated as

$$\boxed{N_A = (4.9 \pm 0.4) \cdot 10^{23}\,\frac{1}{\mathrm{mol}}} \tag{6.8}$$

6.2 Analysis of the Data

Several observations can be made regarding the quality and consistency of the obtained results. Regarding the FWHM, it is noteworthy that the values for α_1 and α_2 as well as for β_1 and β_2 are statistically compatible with one another, suggesting that the first- and second-order measurements yield comparable peak widths. A notable discrepancy is observed in the lattice constant results for NaCl. The β-lines consistently yield lower-quality results than the α-lines. All values overestimate the literature value for the lattice constant, and the β-lines exhibit both lower accuracy and lower precision, as reflected in their higher uncertainties. This behavior was anticipated when examining the expected wavelengths, since all of them are smaller than the literature values for Mo, indicating a systematic offset that particularly affects the β-line measurements. A similar trend is observed in the calculation of Avogadro's number. When using the mean lattice constant – which includes all four K-lines – the standard deviation amounts to 2.60σ. However, when the calculation is restricted to the α_1-line, the standard deviation drops to only 0.48σ:

$$\boxed{N_{A\alpha} = (5.8 \pm 0.4) \cdot 10^{23}\,\frac{1}{\text{mol}}} \tag{6.9}$$

This substantial improvement suggests the presence of a systematic problem that disproportionately affects the β-line measurements, likely linked to the aforementioned discrepancies in the expected wavelengths.

One possible explanation is that the resolution of the NaCl spectrum was relatively low. Increasing the gate time during measurement could potentially resolve this systematic issue.

6.3 Criticism

Some of the uncertainties obtained in this experiment are notably small, implying a high degree of precision. However, this apparent precision is likely unrealistic, as several error sources were neglected in the analysis. In particular, the actual measurement precision of the apparatus itself was not taken into account. Instead, the uncertainty was estimated solely on the basis of the number of digits displayed on the instrument, with the simplifying assumption that this corresponds to the readout error divided by two, as well as statistical errors. Additional contributing factors, such as mechanical positioning accuracy of the goniometer, temperature drift, and counting statistics at low count rates, were omitted entirely and may lead to an underestimation of the true uncertainties.

However, this would not affect the accuracy of the results, as all calculated values were already statistically consistent and reflect[1] the theory quite well, and would continue to do so under a more rigorous uncertainty calculation.

Nevertheless, an improvement of the measurement apparatus would increase the accuracy as well. Similarly, conducting additional measurements would further enhance the reliability of the results.

[1] pun intended

7. Python

The entire Python code can be found on my GitHub at https://github.com/FinnZeumer/PAP-2 [Fin26a]. Additionally, the source code for this project itself is also available there, should anyone be interested in viewing it.

The code is primarily self-written. However, the built-in "MS Copilot" function in Visual Studio Code is used in parts. Unless otherwise indicated, it is used as a support tool and not to generate entire methods. Furthermore, we have a shared folder within our friend group where we share all of our code projects.

PAP 2.2 – Python-Auswertung

Wichtige Funktionen für das PAP.

Alle in diesem Dokument geschriebenen Zeilen Code sind von mir eigenständig geschrieben. Für ein sauberes und struktiriertes Dokument wurde eine eigene Pythonlibary geschrieben, welche insbesondere ans PAP angepasst wurde. Dieses ist unter dem Namen "Papulator" auf meinem GitHub zu finden. Diese soll setig ausgearbeitet werden. Der Stand dieses Protokolls ist der 29.04.2026.

Inhaltsverzeichnis zur Auswertung

Definition der Versuchsvariablen

Im Folgenden sind Versuchsvariablen definiert, die für einen besseren Workflow sorgen sollen. Diese werden zum exportieren und überschreiben von Dateien wichtig sein und ermöglichen es, diese Datei für jeden Versuch zu benutzen und lediglich die Variablen zu verändern. Es ist jedoch empfohlen eine Kopie der Forlage für jeden Versuch zu machen, damit dieses Dokument strukturiert bleibt.

Zudem sind wichtige Konstanten definiert, die immer wieder auftauchen.

In [950…

```
experiment_number = "255"
experiment_name = "Versuchs_Name"
task = "0z"
plt_save_folder = f"../img/plots/"
```

Import aller genutzen Libaries

In [951…

```
# Eigene Pythonlibary für das Pap
import Papulator as pap
from Papulator import Colors as c
from Papulator import const
from Papulator import Sympy_Symbols as sym

# Numpy für bessere Berechnungen
import numpy as np
from numpy import exp, sqrt, log, pi
from uncertainties import unumpy as unp

# Weiteres für bessere Rechnungen
import pylab as py

import math

# Berechnungen und Plotting
from scipy import odr
import scipy.optimize
from scipy.optimize import curve_fit
from scipy.stats import norm
from scipy.stats import chi2
from scipy.stats import poisson
from scipy.signal import find_peaks
from scipy.signal import argrelextrema, argrelmin, argrelmax
from scipy.special import factorial
from scipy.integrate import quad

import matplotlib.pyplot as plt
import matplotlib.mlab as mlab
import matplotlib.transforms as transforms

import astropy.units as u

# Zum Auslesen von Dateien und ähnlichem
import os
import os.path

import pandas as pd
import csv
import re

from io import StringIO
```

```
# Besseres Funktionen handling
import sympy as sp
from sympy import separatevars
```

Auswertung der Aufgaben

Aufgabe 1

In [952…

```
# Import txt Data
angle_T1, counts_T1 = np.loadtxt('data/T1.txt', dtype=str, unpack=True)
angle_T1 = np.char.replace(angle_T1, ',', '.').astype(float)
err_angle_T1 = 0.05 # deg
counts_T1 = np.char.replace(counts_T1, ',', '.').astype(float)
err_counts_T1 = sqrt(counts_T1)
```

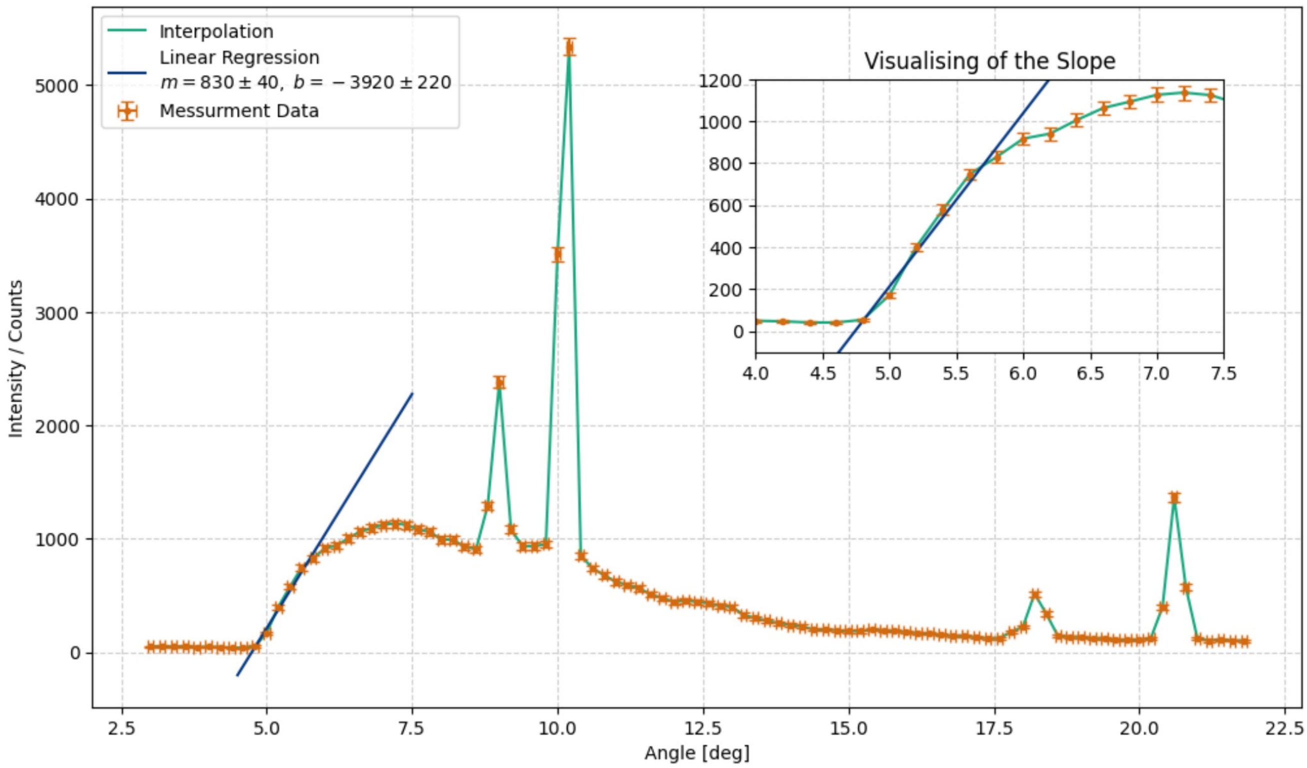
In [953…

```
# Linear regression
def linear(x,m,b):
    return m * x + b
```

In [954…

```
task = '1a'
plt.figure(figsize = (12,7))

# Default interpolation
plt.plot(
    angle_T1,
    counts_T1,
    label='Interpolation',
    )

# Messurment data
plt.errorbar(
    angle_T1,
    counts_T1,
    err_counts_T1,
    err_angle_T1,
    label='Messurment Data',
    marker = '.',
    ls = ' ',
    capsize=3.5
    )

# Regression
# Range of linear regression
start, end = 9, 15
popt, pcov = curve_fit(linear, angle_T1[start:end], counts_T1[start:end], sigma=err_counts_T1[start:end])

slope_T1 = popt[0]
ordinate_T1 = popt[1]

# Extract uncertainties
perr = np.sqrt(np.diag(pcov))

err_slope_T1 = perr[0]
err_ordinate_T1 = perr[1]

x = np.linspace(4.5, 7.5, 100)
plt.plot(
    x,
    linear(x, *popt),
    label = f'Linear Regression\n$m={pap.round_sig_digs(err_slope_T1, slope_T1)[2]}, \\; b={pap.round_sig_digs(err_ordinate_T1, ordinate_T1)[2]}$'
)

plt.legend(loc = 'upper left')
plt.xlabel('Angle [deg]')
plt.ylabel('Intensity / Counts')
plt.grid(alpha = 0.6)
# Small Window
a = plt.axes([.55, .50, .3, .3], facecolor = 'white')

plt.plot(
    angle_T1,
    counts_T1,
    label='Interpolation',
    )

# Messurment data
plt.errorbar(
    angle_T1,
    counts_T1,
    err_counts_T1,
    label='Messurment Data',
    marker = '.',
    ls = ' ',
    capsize=3.5
    )

plt.plot(
    x,
    linear(x, *popt),
    label = f'Linear Regression\n$m={popt[0]:.2f}, \\; b={popt[1]:.2f}$'
)
```

```python
plt.xlim(4, 7.5)
plt.ylim(-100, 1200)
plt.title('Visualising of the Slope')
plt.grid(alpha = 0.6)

plt.savefig(f'{plt_save_folder}hohle_spectrum_and_slope_{task}.pdf')
plt.show()

underground = counts_T1[start]
err_underground = sqrt(underground)
print(f'The underground is {underground} couts')
```




```
The underground is 53.6 couts
```

In [955…

```python
# For better structure, here are the params of the slope printed out:
print(f'Corsssection with the Ordinate: b = {pap.round_sig_digs(err_ordinate_T1, ordinate_T1)[2]}')
print(f'Slope: m = {pap.round_sig_digs(err_slope_T1, slope_T1)[2]}')

m_T1_rnd = pap.round_sig_digs(err_slope_T1, slope_T1)[0]
err_m_T1_rnd = pap.round_sig_digs(err_slope_T1, slope_T1)[1]

b_T1_rnd = pap.round_sig_digs(err_ordinate_T1, ordinate_T1)[0]
err_b_T1_rnd = pap.round_sig_digs(err_ordinate_T1, ordinate_T1)[1]
```

```
Corsssection with the Ordinate: b = -3920 \pm 220
Slope: m = 830 \pm 40
```

In [956…

```python
# Formula for calucation the critical angle
crit_angle = - sym.b / sym.m

values = np.array([
    b_T1_rnd, err_b_T1_rnd,
    m_T1_rnd, err_m_T1_rnd,
])

temp = pap.do_it(crit_angle, [sym.b, sym.m], values, print_formula=False)
val = temp[:,0][0]
err = temp[:,1][0]
rounded = pap.round_sig_digs(err,val)

val_crit_angle = rounded[0]
err_crit_angle = rounded[1]

print(f'The critical angle is: ({rounded[2]})°')
```

```
The critical angle is: (4.7 \pm 0.3)°
```

In [957…

```python
# Next the critical wavelength is being calucated

# constants in use
d_LiF = 201.4 #* 1e-12 #piko to just meter

# converting angel:
val_crit_angle_rad = float(val_crit_angle) * np.pi / 180
err_crit_angle_rad = float(err_crit_angle) * np.pi / 180


#Formula for the critical wavelength
crit_wave = 2 * sym.d * sp.sin(sym.x)  / sym.n

# values to set into the formula:
values = np.array([
    val_crit_angle_rad, err_crit_angle_rad,
    d_LiF,
    1
])
```

```
temp = pap.do_it(crit_wave, [sym.x, sym.d, sym.n], values, [sym.d, sym.n], print_formula=False)

val = temp[:,0][0]
err = temp[:,1][0]
rounded = pap.round_sig_digs(err,val)

val_crit_wave = (rounded[0])
err_crit_wave = (rounded[1])

print(f'The critical wavelength is: ({rounded[2]}) pm')
```

```
The critical wavelength is: (33.0 \pm 2.1) pm
```

In [958…

```
# Calculation planks kosntant

crit_lambda_sym = sp.symbols(r'\lambda_\text{{crit}}')

# Formula
planks_T1 = crit_lambda_sym * sym.e * sym.U / sym.c

# constans/values
U_B = 35 * 1e3  # V
err_U_B = 0.05 * 1e3 # V

lams = float(val_crit_wave) * 1e-12
err_lams = float(err_crit_wave) * 1e-12

values = np.array([
    lams, err_lams,
    U_B, err_U_B,
    const.c,
    const.e
])

temp = pap.do_it(planks_T1, [crit_lambda_sym, sym.U, sym.c, sym.e], values, [sym.c, sym.e], False)

# For plank we expect dimension of 10^-34
val = temp[:,0][0] * 1e34
err = temp[:,1][0] * 1e34

rounded = pap.round_sig_digs(err,val)

val_h_T1 = rounded[0]
err_h_T1 = rounded[1]

print(f'The planks constant is is: ({rounded[2]}) * 1e-34 J \\ s')

print(f'Its standard diviation to the lit value is: {pap.std_abw(float(val_h_T1), const.h * 1e34, float(err_h_T1))} sigma')
```

```
The planks constant is is: (6.2 \pm 0.4) * 1e-34 J \ s
Its standard diviation to the lit value is: 1.07 sigma
```

In [959…

```
#Formula for the critical wavelength
crit_wave = 2 * sym.d * sp.sin(sym.x)  / sym.n

# values to set into the formula:
values = np.array([
    val_crit_angle_rad, err_crit_angle_rad,
    d_LiF,
    2
])

temp = pap.do_it(crit_wave, [sym.x, sym.d, sym.n], values, [sym.d, sym.n], print_formula=False)

val = temp[:,0][0]
err = temp[:,1][0]
rounded = pap.round_sig_digs(err,val)

val_crit_wave_2Ord = float(rounded[0])
err_crit_wave_2Ord = float(rounded[1])

print(f'The critical wavelength for the second order is: ({rounded[2]}) pm')
```

```
The critical wavelength for the second order is: (16.5 \pm 1.1) pm
```

In [960…

```
# Expected angle of the second peak

d_LiF = 201.4 #* 1e-12 #piko to just meter
second_peak_angle = sp.asin(((2 * crit_lambda_sym / (2*  sym.d)))) / (sp.pi / 180)

values = np.array([
    val_crit_wave, err_crit_wave,
    d_LiF
])

temp = pap.do_it(second_peak_angle, [crit_lambda_sym, sym.d], values, [sym.d], False)

val = temp[:,0][0]
err = temp[:,1][0]

rounded = pap.round_sig_digs(err,val)

val_second_angle = rounded[0]
err_second_angle = rounded[1]

print(f'The next peak starts as angle of: ({rounded[2]})°')
```

```
The next peak starts as angle of: (9.4 \pm 0.6)°
```

Aufgabe 2

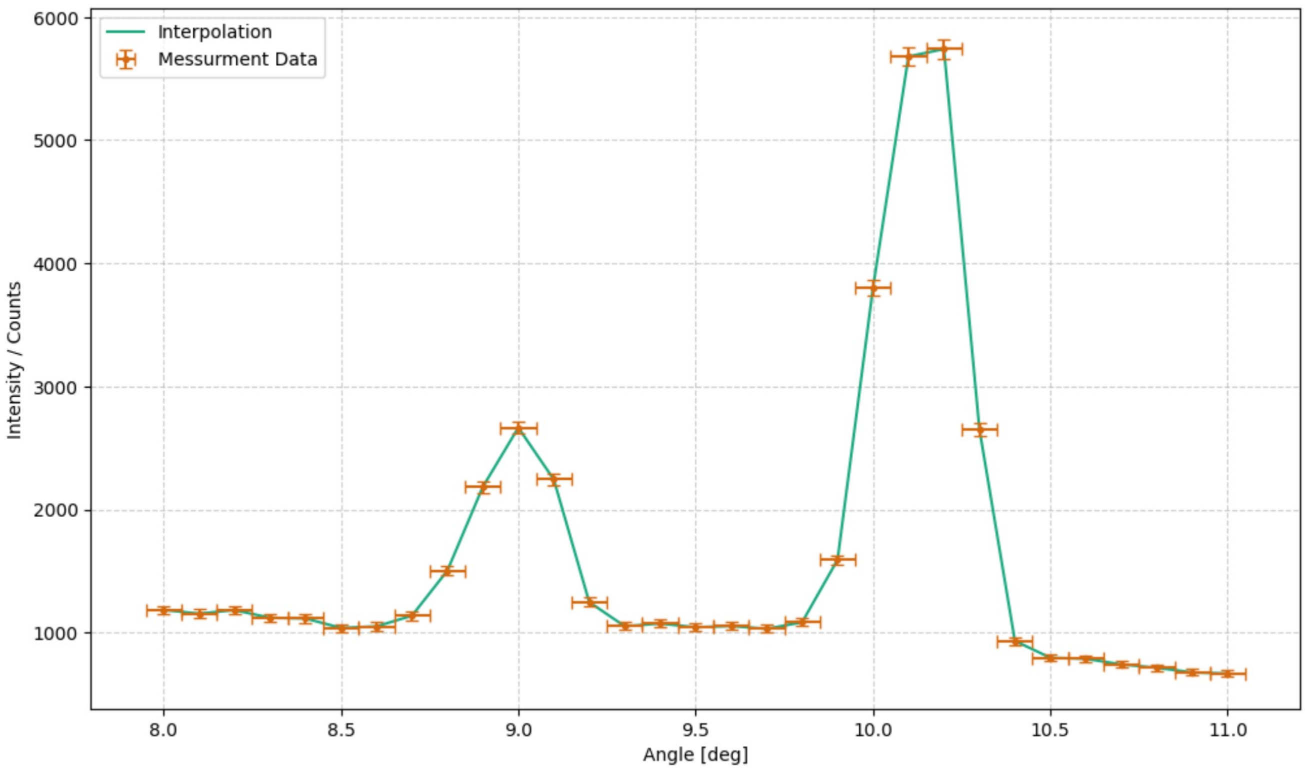
First Order

In [961…

```
# Import txt Data
angle_1ord, counts_1ord = np.loadtxt('data/T2_1O.txt', dtype=str, unpack=True)
angle_1ord = np.char.replace(angle_1ord, ',', '.').astype(float)
err_angle_1ord = 0.05 # deg
counts_1ord = np.char.replace(counts_1ord, ',', '.').astype(float)
err_counts_1ord = sqrt(counts_1ord)
```

In [962…

```
task = '2a'
plt.figure(figsize = (12,7))

# Default interpolation
plt.plot(
    angle_1ord,
    counts_1ord,
    label='Interpolation',
    )

# Messurment data
plt.errorbar(
    angle_1ord,
    counts_1ord,
    err_counts_1ord,
    err_angle_1ord,
    label='Messurment Data',
    marker = '.',
    ls = ' ',
    capsize=3.5
    )

plt.legend(loc = 'upper left')
plt.xlabel('Angle [deg]')
plt.ylabel('Intensity / Counts')
plt.grid(alpha = 0.6)

plt.savefig(f'{plt_save_folder}first_order_LiF_{task}.pdf')
plt.show()
```



In [963…

```
# find extrima
peaks, _ = find_peaks(counts_1ord, height=2500)
n_max_alpha = counts_1ord[peaks[1]]
deg_max_alpha = angle_1ord[peaks[1]]
n_max_beta = counts_1ord[peaks[0]]
deg_max_beta = angle_1ord[peaks[0]]
```

In [964…

```
# function
wavlength_by_ord  = 2 / sym.n * sym.d * sp.sin(sym.x)

degs = np.array([deg_max_alpha, deg_max_beta])
print(degs)
values = np.column_stack([
    degs * np.pi / 180, [0.05 * np.pi / 180] * 2,
    [1] * 2,
    [d_LiF] * 2
])

temp = pap.do_it(wavlength_by_ord, [sym.x, sym.n, sym.d], values, [sym.n, sym.d], False)
```

```python
# print(temp)
# print(temp[:,0])

vals = temp[:,0]
err = temp[:,1]

alpha_1 = vals[0]
err_alpha_1 = err[0]

beta_1 = vals[1]
err_beta_1 = err[1]

print(f'Wavelength of alpha order 1: {pap.round_sig_digs(err_alpha_1, alpha_1)[2]}')
print(f'Wavelength of beta order 1: {pap.round_sig_digs(err_beta_1, beta_1)[2]}')
```

```
[10.2  9. ]
Wavelength of alpha order 1: 71.3 \pm 0.3
Wavelength of beta order 1: 63.0 \pm 0.3
```

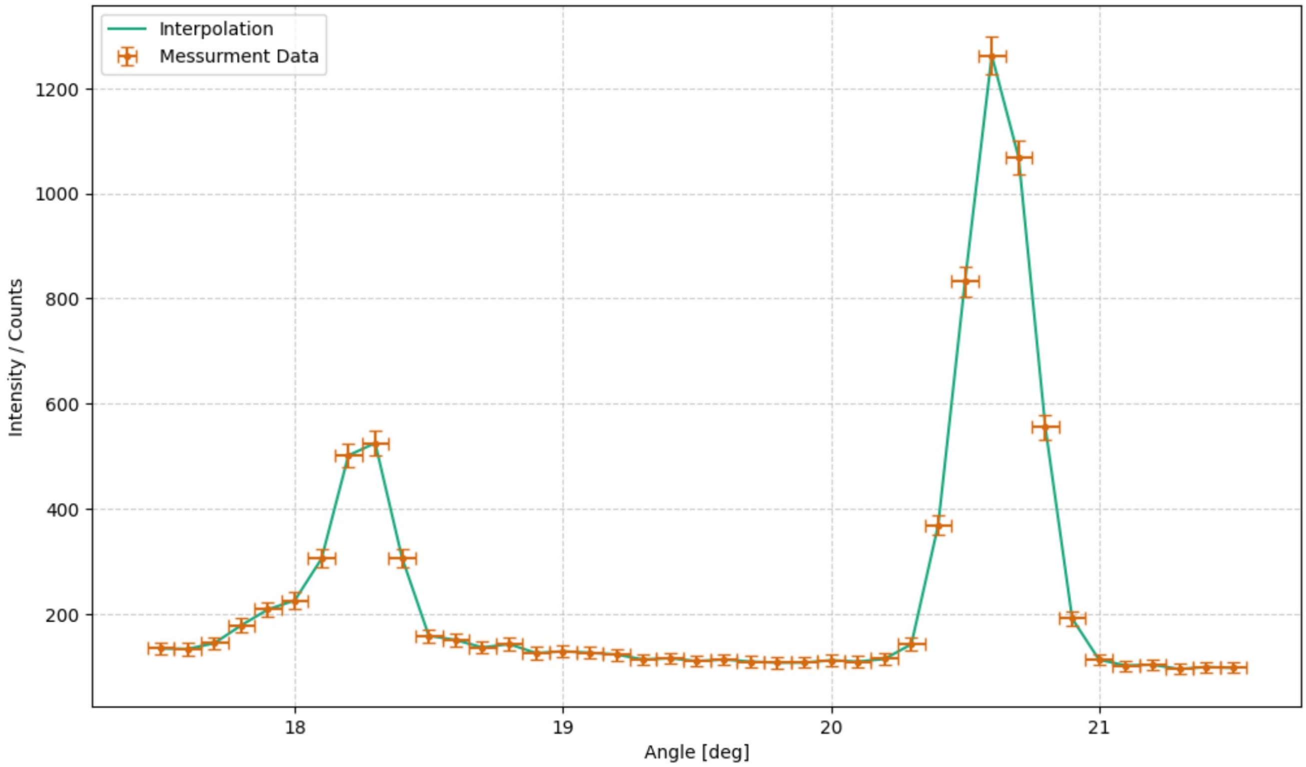
Second Order

In [965…

```python
# Import txt Data
angle_2ord, counts_2ord = np.loadtxt('data/T2_2O.txt', dtype=str, unpack=True)
angle_2ord = np.char.replace(angle_2ord, ',', '.').astype(float)
err_angle_2ord = 0.05 # deg
counts_2ord = np.char.replace(counts_2ord, ',', '.').astype(float)
err_counts_2ord = sqrt(counts_2ord)
```

In [966…

```python
task = '2b'
plt.figure(figsize = (12,7))

# Default interpolation
plt.plot(
    angle_2ord,
    counts_2ord,
    label='Interpolation',
    )

# Messurment data
plt.errorbar(
    angle_2ord,
    counts_2ord,
    err_counts_2ord,
    err_angle_2ord,
    label='Messurment Data',
    marker = '.',
    ls = ' ',
    capsize=3.5
    )

plt.legend(loc = 'upper left')
plt.xlabel('Angle [deg]')
plt.ylabel('Intensity / Counts')
plt.grid(alpha = 0.6)

plt.savefig(f'{plt_save_folder}second_order_LiF_{task}.pdf')
plt.show()
```



In [967…

```python
# find extrima
peaks_2, _ = find_peaks(counts_2ord, height=400)
n_max_alpha_2 = counts_2ord[peaks_2]
deg_max_alpha_2 = angle_2ord[peaks_2[1]]
n_max_beta_2 = counts_2ord[peaks_2[0]]
deg_max_beta_2 = angle_2ord[peaks_2[0]]
```

```
print(deg_max_alpha_2, deg_max_beta_2)
```

20.6 18.3

In [968…

```
# function
wavlength_by_ord  = 2 / sym.n * sym.d * sp.sin(sym.x)

degs = np.array([deg_max_alpha_2, deg_max_beta_2])
print(degs)

values = np.column_stack([
    degs * np.pi / 180, [0.05 * np.pi / 180] * 2,
    [2] * 2,
    [d_LiF] * 2
])

temp = pap.do_it(wavlength_by_ord, [sym.x, sym.n, sym.d], values, [sym.n, sym.d], False)
# print(temp)
# print(temp[:,0])

vals = temp[:,0]
err = temp[:,1]

alpha_2 = vals[0]
err_alpha_2 = err[0]

beta_2 = vals[1]
err_beta_2 = err[1]

print(f'Wavelength of alpha order 2: ({pap.round_sig_digs(err_alpha_2, alpha_2)[2]}) pm')
print(f'Wavelength of beta order 2: ({pap.round_sig_digs(err_beta_2, beta_2)[2]} pm)')
```

```
[20.6 18.3]
Wavelength of alpha order 2: (70.86 \pm 0.16) pm
Wavelength of beta order 2: (63.24 \pm 0.17 pm)
```

In [992…

```
# averaging:
Mo_wavelength_alpha = (alpha_2 + alpha_1) / 2
err_Mo_wavelength_alpha = sqrt(err_alpha_2**2 + err_alpha_1**2)

Mo_wavelength_beta = (beta_2 + beta_1) / 2
err_Mo_wavelength_beta = sqrt(err_beta_2**2 + err_beta_1**2)

print(f'The wavelength of Mo for alpha is: ({pap.round_sig_digs(err_Mo_wavelength_alpha ,Mo_wavelength_alpha)[2]}) pm')
print(f'The wavelength of Mo for beta is: ({pap.round_sig_digs(err_Mo_wavelength_beta ,Mo_wavelength_beta)[2]}) pm')
```

```
The wavelength of Mo for alpha is: (71.1 \pm 0.4) pm
The wavelength of Mo for beta is: (63.1 \pm 0.4) pm
```

In [970…

```
Mo_Kalpha_lit = 71.1 # pm
Mo_Kbeta_lit = 63.1 # pm

sig_diveration_alpha = pap.std_abw(Mo_Kalpha_lit, Mo_wavelength_alpha, err_Mo_wavelength_alpha)
sig_diveration_beta = pap.std_abw(Mo_Kbeta_lit, Mo_wavelength_beta, err_Mo_wavelength_beta)

print(f'The standard diviation from the litature value of the K Alpha line:\n{sig_diveration_alpha}')
print(f'The standard diviation from the litature value of the K Beta line:\n{sig_diveration_beta}')
print('Those are only true for the non rounded values!')
```

```
The standard diviation from the litature value of the K Alpha line:
0.01
The standard diviation from the litature value of the K Beta line:
0.06
Those are only true for the non rounded values!
```

In [971…

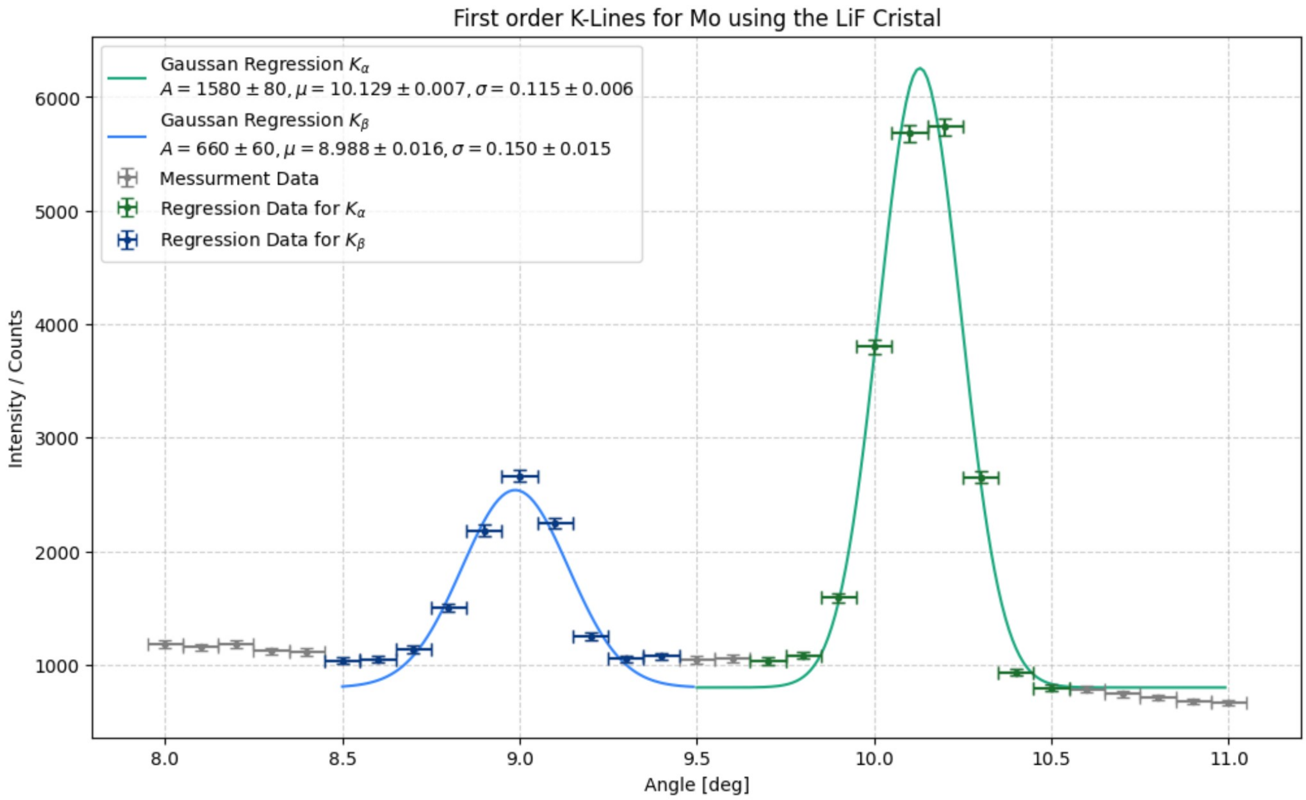
```
def gauss(x, A, mu, sig):
    return A / (sqrt(2 * pi) * sig) * exp(- (x - mu) ** 2 / (2 * sig ** 2)) + 800
```

In [972…

```
# Finding the HWFM of the K alpha line in first order
task = '2c'
plt.figure(figsize = (12,7))

# Regressionfor K Alpha
a_start, a_end = 17, -5

p0_alpha = [5000, 10, 0.4]
popt_alpha, pcov_alpha = curve_fit(gauss, angle_1ord[a_start:a_end], counts_1ord[a_start:a_end], p0_alpha, sigma = err_counts_1ord[a_start:a_end])
degx_alpha = np.arange(9.5,11, 0.01)

perr_alpha = np.sqrt(np.diag(pcov_alpha))

A_alpha = popt_alpha[0]
err_A_alpha = perr_alpha[0]

mu_alpha = popt_alpha[1]
err_mu_alpha = perr_alpha[1]

sig_alpha = popt_alpha[2]
err_sig_alpha = perr_alpha[2]

# To see the results clearly
print(f"Fitted parameters (Amplitude, Mean, Sigma): {popt_alpha}")
print(f"Errors (Standard Deviations): {perr_alpha}")

plt.plot(
    degx_alpha,
    gauss(degx_alpha, *popt_alpha),
    label = f'Gaussan Regression $K_\\alpha$\n$A={pap.round_sig_digs(err_A_alpha, A_alpha)[2]}, \\mu={pap.round_sig_digs(err_mu_alpha, mu_alpha)[2]}, \\s
    color = c.LIGHT_GREEN
    )
```

```python
# Regressionfor K Beta
b_start, b_end = 5, -16

p0_beta = [5000, 9, 0.4]
popt_beta, pcov_beta = curve_fit(gauss, angle_1ord[b_start:b_end], counts_1ord[b_start:b_end], p0_beta, sigma = err_counts_1ord[b_start:b_end])
degx_beta = np.arange(8.5, 9.5, 0.01)

perr_beta = np.sqrt(np.diag(pcov_beta))

A_beta = popt_beta[0]
err_A_beta = perr_beta[0]

mu_beta = popt_beta[1]
err_mu_beta = perr_beta[1]

sig_beta = popt_beta[2]
err_sig_beta = perr_beta[2]

plt.plot(
    degx_beta,
    gauss(degx_beta, *popt_beta),
    label = f'Gaussan Regression $K_\\beta$\n$A={pap.round_sig_digs(err_A_beta, A_beta)[2]}, \\mu={pap.round_sig_digs(err_mu_beta, mu_beta)[2]}, \\sigma=
    color = c.LIGHT_BLUE
    )

# Messurment data
plt.errorbar(
    angle_1ord,
    counts_1ord,
    err_counts_1ord,
    err_angle_1ord,
    label='Messurment Data',
    marker = '.',
    ls = ' ',
    capsize=3.5,
    color = 'gray'
    )

# Messurment data used for regression Alpha
plt.errorbar(
    angle_1ord[a_start:a_end],
    counts_1ord[a_start:a_end],
    err_counts_1ord[a_start:a_end],
    err_angle_1ord,
    label='Regression Data for $K_\\alpha$',
    marker = '.',
    ls = ' ',
    capsize=3.5,
    color= c.GREEN
    )

# Messurment data used for regression Alpha
plt.errorbar(
    angle_1ord[b_start:b_end],
    counts_1ord[b_start:b_end],
    err_counts_1ord[b_start:b_end],
    err_angle_1ord,
    label='Regression Data for $K_\\beta$',
    marker = '.',
    ls = ' ',
    capsize=3.5,
    color= c.BLUE
    )

plt.title(f'First order K-Lines for Mo using the LiF Cristal')
plt.legend(loc = 'upper left')
plt.xlabel('Angle [deg]')
plt.ylabel('Intensity / Counts')
plt.grid(alpha = 0.6)

plt.savefig(f'{plt_save_folder}first_order_LiF_FWHM{task}.pdf')
plt.show()
```

```
Fitted parameters (Amplitude, Mean, Sigma): [1.57660646e+03 1.01289766e+01 1.15310484e-01]
Errors (Standard Deviations): [8.30446611e+01 6.80407916e-03 5.75718969e-03]
```

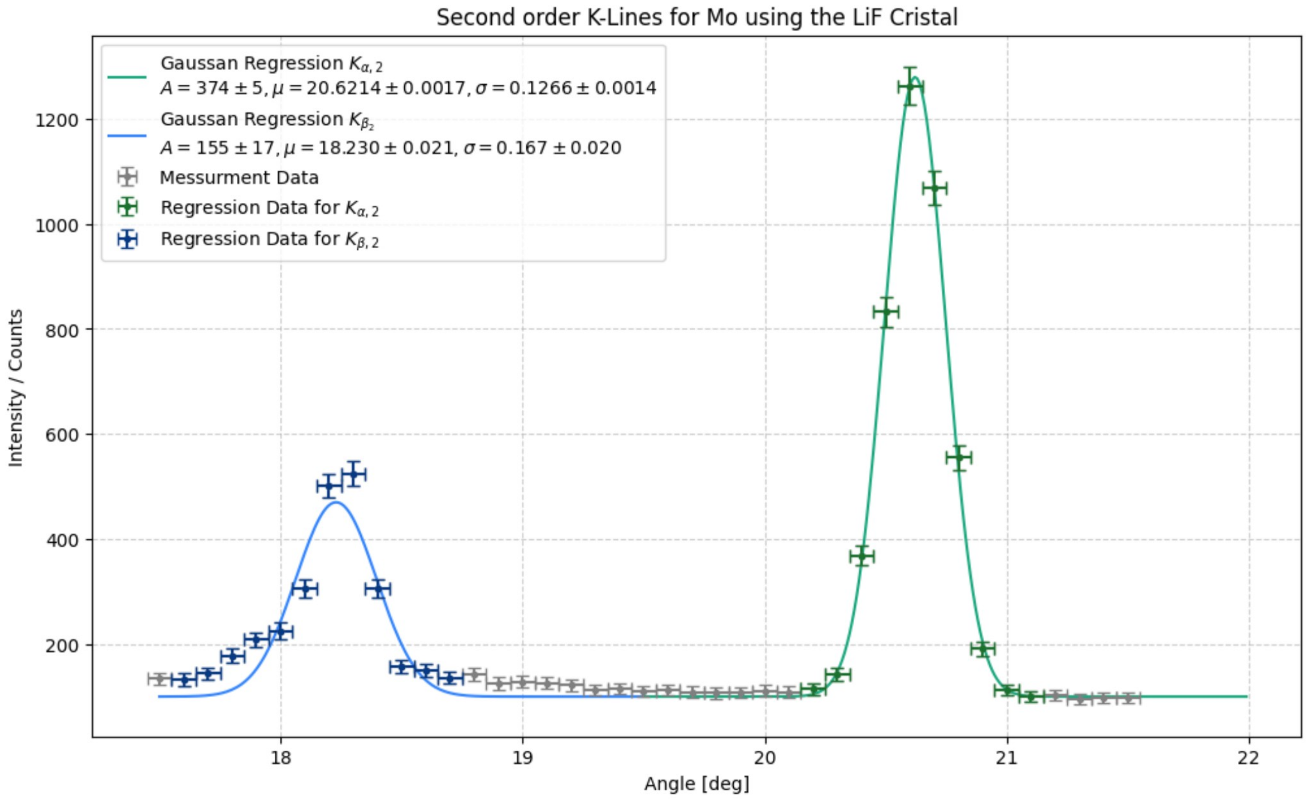


In [973…

```python
def gauss(x, A, mu, sig):
    return A / (sqrt(2 * pi) * sig) * exp(- (x - mu) ** 2 / (2 * sig ** 2)) + 100

# Finding the HWFM of the K alpha_2 line in second order
task = '2d'
plt.figure(figsize = (12,7))

# Regressionfor K alpha_2
a_start, a_end = 27, -4

p0_alpha_2 = [1000, 20.5, 0.4]
popt_alpha_2, pcov_alpha_2 = curve_fit(gauss, angle_2ord[a_start:a_end], counts_2ord[a_start:a_end], p0_alpha_2, sigma = err_counts_2ord[a_start:a_end])
degx_alpha_2 = np.arange(19.5, 22, 0.01)

perr_alpha_2 = np.sqrt(np.diag(pcov_alpha_2))

A_alpha_2 = popt_alpha_2[0]
err_A_alpha_2 = perr_alpha_2[0]

mu_alpha_2 = popt_alpha_2[1]
err_mu_alpha_2 = perr_alpha_2[1]

sig_alpha_2 = popt_alpha_2[2]
err_sig_alpha_2 = perr_alpha_2[2]

# To see the results clearly
print(f"Fitted parameters (Amplitude, Mean, Sigma): {popt_alpha_2}")
print(f"Errors (Standard Deviations): {perr_alpha_2}")

plt.plot(
    degx_alpha_2,
    gauss(degx_alpha_2, *popt_alpha_2),
    label = 'Gaussan Regression $K_{\\alpha,2}$' + f'\n$A={pap.round_sig_digs(err_A_alpha_2, A_alpha_2)[2]}, \\mu={pap.round_sig_digs(err_mu_alpha_2, mu_
    color = c.LIGHT_GREEN
    )


# Regressionfor K beta_2
b_start, b_end = 1, -28

p0_beta_2 = [400, 18.3, 0.4]
popt_beta_2, pcov_beta_2 = curve_fit(gauss, angle_2ord[b_start:b_end], counts_2ord[b_start:b_end], p0_beta_2, sigma = err_counts_2ord[b_start:b_end])
degx_beta_2 = np.arange(17.5, 19.5, 0.01)

perr_beta_2 = np.sqrt(np.diag(pcov_beta_2))

A_beta_2 = popt_beta_2[0]
err_A_beta_2 = perr_beta_2[0]

mu_beta_2 = popt_beta_2[1]
err_mu_beta_2 = perr_beta_2[1]

sig_beta_2 = popt_beta_2[2]
err_sig_beta_2 = perr_beta_2[2]

# To see the results clearly
print(f"Fitted parameters (Amplitude, Mean, Sigma): {popt_beta_2}")
print(f"Errors (Standard Deviations): {perr_beta_2}")

plt.plot(
    degx_beta_2,
    gauss(degx_beta_2, *popt_beta_2),
    label = 'Gaussan Regression $K_{\\beta_2}$' f'\n$A={pap.round_sig_digs(err_A_beta_2, A_beta_2)[2]}, \\mu={pap.round_sig_digs(err_mu_beta_2, mu_beta_2
```

```python
    color = c.LIGHT_BLUE
    )

# Messurment data
plt.errorbar(
    angle_2ord,
    counts_2ord,
    err_counts_2ord,
    err_angle_2ord,
    label='Messurment Data',
    marker = '.',
    ls = ' ',
    capsize=3.5,
    color = 'gray'
    )

# Messurment data used for regression alpha_2
plt.errorbar(
    angle_2ord[a_start:a_end],
    counts_2ord[a_start:a_end],
    err_counts_2ord[a_start:a_end],
    err_angle_2ord,
    label='Regression Data for $K_{\\alpha,2}$',
    marker = '.',
    ls = ' ',
    capsize=3.5,
    color= c.GREEN
    )

# Messurment data used for regression alpha_2
plt.errorbar(
    angle_2ord[b_start:b_end],
    counts_2ord[b_start:b_end],
    err_counts_2ord[b_start:b_end],
    err_angle_2ord,
    label='Regression Data for $K_{\\beta,2}$',
    marker = '.',
    ls = ' ',
    capsize=3.5,
    color= c.BLUE
    )

plt.title(f'Second order K-Lines for Mo using the LiF Cristal')
plt.legend(loc = 'upper left')
plt.xlabel('Angle [deg]')
plt.ylabel('Intensity / Counts')
plt.grid(alpha = 0.6)

# plt.savefig(f'{plt_save_folder}second_order_LiF_FWHM{task}.pdf')
plt.show()
```

```
Fitted parameters (Amplitude, Mean, Sigma): [3.74224248e+02 2.06213659e+01 1.26597870e-01]
Errors (Standard Deviations): [4.57796265e+00 1.69520709e-03 1.41214204e-03]
Fitted parameters (Amplitude, Mean, Sigma): [154.52573522  18.22985039   0.16666023]
Errors (Standard Deviations): [17.37499378  0.02097676  0.01998355]
```



In [974…

```python
# Calculating the actual FWHM for all for peaks

# Formula
FWHM = 2 * sp.sqrt(2 * sp.ln(2)) * sym.sigma

sigs = np.array([sig_alpha, sig_beta, sig_alpha_2, sig_beta_2])
err_sigs = np.array([err_sig_alpha, err_sig_beta, err_sig_alpha_2, err_sig_beta_2])
values = np.column_stack([
    sigs, err_sigs
```

```
])

temp = pap.do_it(FWHM, [sym.sigma], values, print_formula=False)
vals_FWHM = temp[:,0] #/ pi * 180
errs_FWHM = temp[:,1] #/ pi * 180

element = ['alpha_1', 'beta_1', 'alpha_2', 'beta_2']
for i in range(4):
    print(f'FWHM {element[i]} = ({pap.round_sig_digs(errs_FWHM[i], vals_FWHM[i])[2]})°')
```

```
FWHM alpha_1 = (0.272 \pm 0.014)°
FWHM beta_1 = (0.35 \pm 0.03)°
FWHM alpha_2 = (0.298 \pm 0.003)°
FWHM beta_2 = (0.39 \pm 0.05)°
```

In [975…

```
# # Calculating the actual FWHM for all for peaks

# # Formula
# FWHM_p = sym.mu + (2 * sp.sqrt(2 * sp.ln(2)) * sym.sigma) #/2
# FWHM_m = sym.mu - (2 * sp.sqrt(2 * sp.ln(2)) * sym.sigma) #/2

# FWHM_c = FWHM_p - FWHM_m

# sigs = np.array([sig_alpha, sig_beta, sig_alpha_2, sig_beta_2])
# err_sigs = np.array([err_sig_alpha, err_sig_beta, err_sig_alpha_2, err_sig_beta_2])

# muhs = np.array([mu_alpha, mu_beta, mu_alpha_2, mu_beta_2])
# err_muhs = np.array([err_mu_alpha, err_mu_beta, err_mu_alpha_2, err_mu_beta_2])


# values = np.column_stack([
#     muhs, err_muhs,
#     sigs, err_sigs
# ])

# temp = pap.do_it(FWHM_c, [sym.mu, sym.sigma], values, print_formula=False)
# vals_FWHM = temp[:,0] #/ pi * 180
# errs_FWHM = temp[:,1] #/ pi * 180
# for i in range(4):
#     print(f'FWHM {i} = ({pap.round_sig_digs(errs_FWHM[i], vals_FWHM[i])[2]})°')
```

In [976…

```
def wlange(deg, d, n):
    return 2 / n * d * np.sin(np.deg2rad(deg))
```

In [977…

```
# Halfwidth as wavelength

# a_1, b_1, a_, b_2
element = ['alpha_1', 'beta_1', 'alpha_2', 'beta_2']

for i in range (2):
    n = i + i * 1
    print(f'FWHM {element[n]}: ({pap.round_sig_digs(wlange(errs_FWHM[0:2], d_LiF, 1)[i], wlange(vals_FWHM[0:2], d_LiF, 1)[i])[2]}) pm')
    print(f'FWHM {element[n + 1]}: ({pap.round_sig_digs(wlange(errs_FWHM[2:4], d_LiF, 2)[i], wlange(vals_FWHM[2:4], d_LiF, 2)[i])[2]}) pm')
    print('-' * 15)
```

```
FWHM alpha_1: (1.91 \pm 0.10) pm
FWHM beta_1: (1.048 \pm 0.012) pm
---------------
FWHM alpha_2: (2.49 \pm 0.24) pm
FWHM beta_2: (1.38 \pm 0.17) pm
---------------
```

Aufgabe 3

In [978…

```
# Creating the Array for the voltages
steps = 15
U_B_T3 = np.zeros([steps])
err_U_B_T3 = 0.05 # kV
start_voltage = 20 # kV
for i in range(steps):
    U_B_T3[i] = start_voltage
    start_voltage = start_voltage + 1

# U_B * u.kV
```

In [979…

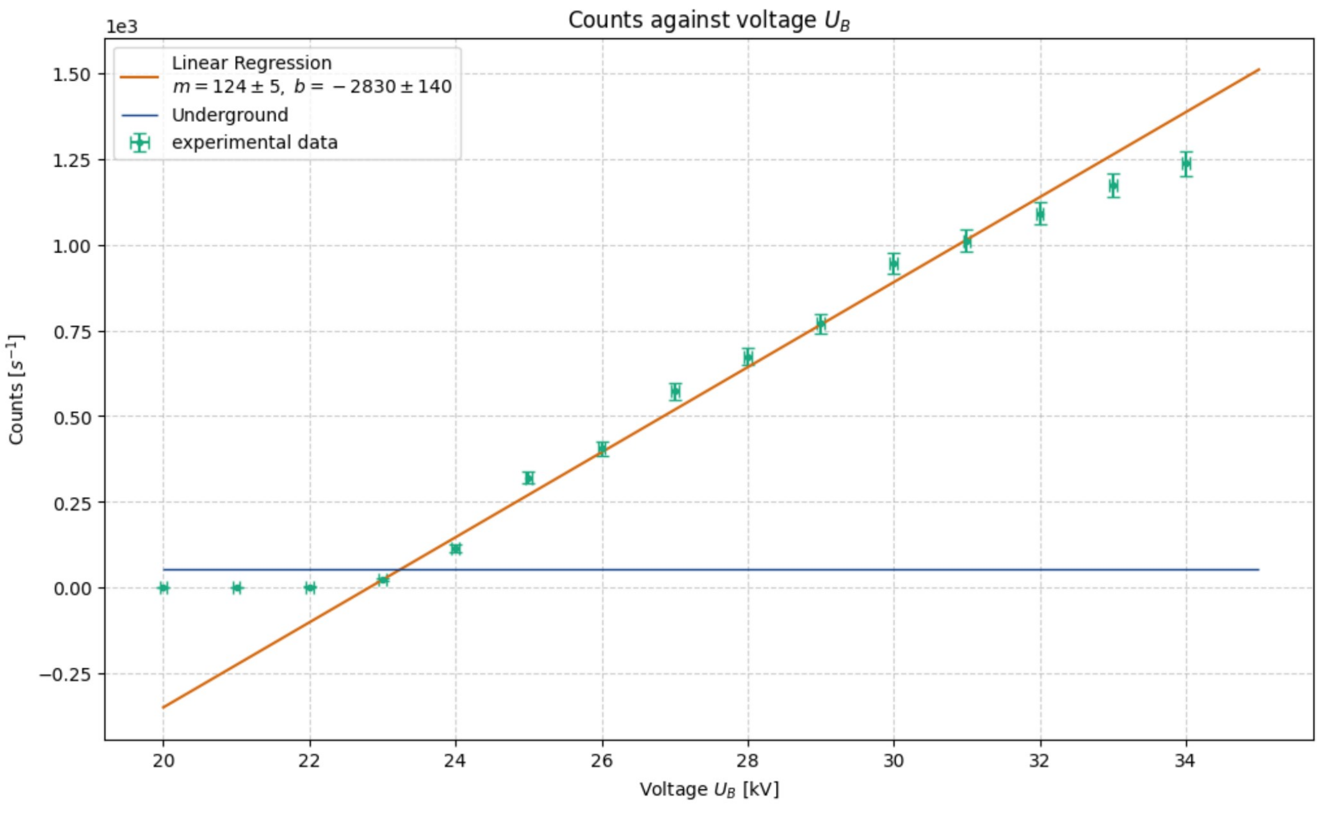
```
# Cretaing the array for the counts for each foltage
counts_T3 = np.array([0.9, 1.05, 1.85, 23.15, 116.2, 321.6, 405.1, 573.1, 673.8, 769.2, 945.8, 1011.4, 1090.8, 1172.5, 1235.7])
# print(f'Same amount of voltages and counts: {len(U_B_T3) == len(counts_T3)}')

# Error form limited display
err_counts_T3 = np.zeros([steps])
for i in range(steps):
    if counts_T3[i] > 100:
        err_counts_T3[i] = 0.05
    else:
        err_counts_T3[i] = 0.005

err_n_T3 = sqrt(err_counts_T3**2 + sqrt(counts_T3)**2)
```

In [980…

```
task = '3a'
plt.figure(figsize = (12,7))

plt.errorbar(
    U_B_T3,
    counts_T3,
    err_n_T3,
    err_U_B_T3,
    marker = '.',
```

```
    ls = ' ',
    label='experimental data',
    capsize= 3.5
)

# Regression
# Range of linear regression
start, end = 4, -1
popt_T4, pcov_T4 = curve_fit(linear, U_B_T3[start:end], counts_T3[start:end], sigma=err_n_T3[start:end])

slope_T4 = popt_T4[0]
ordinate_T4 = popt_T4[1]

# Extract uncertainties
perr = np.sqrt(np.diag(pcov_T4))

err_slope_T4 = perr[0]
err_ordinate_T4 = perr[1]

x = np.linspace(20, 35, 1000)
plt.plot(
    x,
    linear(x, *popt_T4),
    label = f'Linear Regression\n$m={pap.round_sig_digs(err_slope_T4, slope_T4)[2]}, \\; b={pap.round_sig_digs(err_ordinate_T4, ordinate_T4)[2]}$'
)

plt.hlines(
    [underground],
    20,
    35,
    lw = 1,
    colors= c.BLUE,
    label='Underground'
)

print(sqrt(underground))

pap.plot_me('Counts against voltage $U_B$', 'Voltage $U_B$ [kV]', 'Counts [$s^{-1}$]', f'{plt_save_folder}Against_U_B_{task}')
```

7.321202087089251



Out[980… <Axes: title={'center': 'Counts against voltage U_B'}, xlabel='Voltage U_B [kV]', ylabel='Counts [s^{-1}]'>

In [981…
```
val = np.arange(22.3, 25, 0.001)
for i in range(0, 1000):
    if linear(val[i], *popt_T4) >= underground:
        print('Hello')
        U_E = val[i] #kV
        break
for i in range(0, 10000):
    if linear(val[i], *popt_T4) >= underground - err_underground:
        print('Here')
        err_U_E = np.abs(U_E - val[i])
        break

print(f'Einsatzspannung U_E = ({pap.round_sig_digs(err_U_E, U_E)[2]}) kV')
```

Hello
Here
Einsatzspannung U_E = (23.25 \pm 0.06) kV

In [982…
```
# Finding Plack's constant (once again)
lam_sym = sp.symbols('\\lambda')

h_2 = lam_sym * sym.U * sym.e / sym.c
```

```python
beta_here = 7.5
beta_lam =  wlange(beta_here, d_LiF, 1)

err_beta_here = 0.05
err_beta_lam =  wlange(err_beta_here, d_LiF, 1)

values = np.column_stack([
    beta_lam * 1e-12, err_beta_lam * 1e-12,
    U_E * 1e3, err_U_E * 1e3,
    const.e,
    const.c
])

temp = pap.do_it(h_2, [lam_sym, sym.U, sym.e, sym.c], values, [sym.e, sym.c], False)

val_h_2 = temp[:,0][0] * 1e34
err_h_2 = temp[:,1][0] * 1e34

print(f'The plancks constant is: h = ({pap.round_sig_digs(err_h_2, val_h_2)[2]}) *10^-34 J s ')

std_div_h2 = pap.std_abw(val_h_2, const.h * 1e34, err_h_2)
print(f'The standard diviation to the lit value is: {std_div_h2} sigma')
```

```
The plancks constant is: h = (6.53 \pm 0.05) *10^-34 J s
The standard diviation to the lit value is: 2.02 sigma
```

Aufgabe 4

In [983…

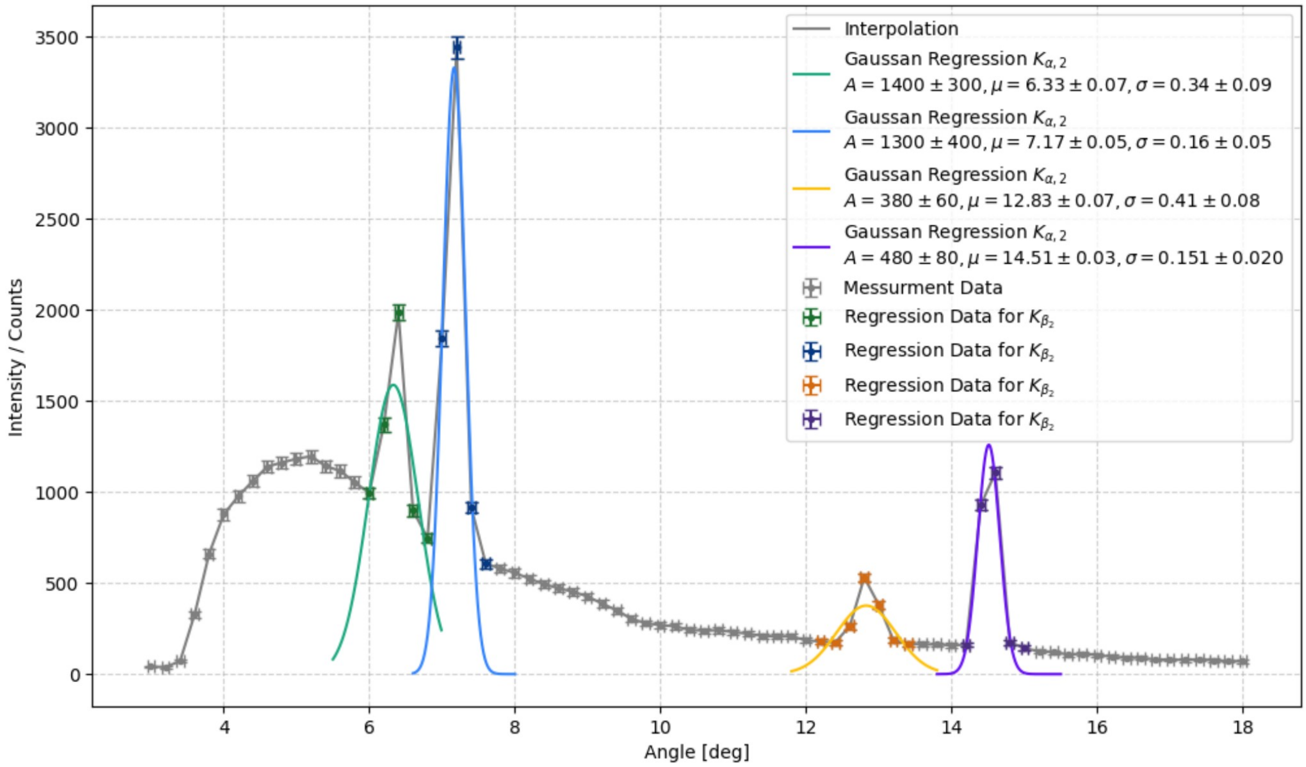
```python
# Import txt Data
angle_T4, counts_T4 = np.loadtxt('data/T4.txt', dtype=str, unpack=True)
angle_T4 = np.char.replace(angle_T4, ',', '.').astype(float)
err_angle_T4 = 0.05 # deg
counts_T4 = np.char.replace(counts_T4, ',', '.').astype(float)
err_counts_T4 = sqrt(counts_T4)
```

In [984…

```python
task = '4a'
plt.figure(figsize = (12,7))

def gauss(x, A, mu, sig):
    return A / (sqrt(2 * pi) * sig) * exp(- (x - mu) ** 2 / (2 * sig ** 2))

# Default interpolation
plt.plot(
    angle_T4,
    counts_T4,
    label='Interpolation',
    color = 'gray'
    )

# Messurment data
plt.errorbar(
    angle_T4,
    counts_T4,
    err_counts_T4,
    err_angle_T4,
    label='Messurment Data',
    marker = '.',
    ls = ' ',
    capsize=3.5,
    color = 'gray'
    )

# Regressionfor K beta
a_start, a_end = 15, 20

p0_beta_T4 = [1800, 6.4, 0.4]
popt_beta_T4, pcov_beta_T4 = curve_fit(gauss, angle_T4[a_start:a_end], counts_T4[a_start:a_end], p0_beta_T4, sigma = err_counts_T4[a_start:a_end])
degx_beta_T4 = np.arange(5.5, 7, 0.01)

perr_beta_T4 = np.sqrt(np.diag(pcov_beta_T4))

A_beta_T4 = popt_beta_T4[0]
err_A_beta_T4 = perr_beta_T4[0]

mu_beta_T4 = popt_beta_T4[1]
err_mu_beta_T4 = perr_beta_T4[1]

sig_beta_T4 = popt_beta_T4[2]
err_sig_beta_T4 = perr_beta_T4[2]

# Messurment data used for regression Alpha
plt.errorbar(
    angle_T4[a_start:a_end],
    counts_T4[a_start:a_end],
    err_counts_T4[a_start:a_end],
    err_angle_T4,
    label='Regression Data for $K_{\\beta_2}$',
    marker = '.',
    ls = ' ',
    capsize=3.5,
    color= c.GREEN
    )

plt.plot(
    degx_beta_T4,
    gauss(degx_beta_T4, *popt_beta_T4),
    label = 'Gaussan Regression $K_{\\alpha,2}$' + f'\n$A={pap.round_sig_digs(err_A_beta_T4, A_beta_T4)[2]}, \\mu={pap.round_sig_digs(err_mu_beta_T4, mu_
```

```python
    color = c.LIGHT_GREEN
    )

# Regressionfor K alpha
a_start, a_end = 20, 24

p0_alpha_T4 = [1800, 6.4, 0.4]
popt_alpha_T4, pcov_alpha_T4 = curve_fit(gauss, angle_T4[a_start:a_end], counts_T4[a_start:a_end], p0_alpha_T4, sigma = err_counts_T4[a_start:a_end])
degx_alpha_T4 = np.arange(6.6, 8, 0.01)

perr_alpha_T4 = np.sqrt(np.diag(pcov_alpha_T4))

A_alpha_T4 = popt_alpha_T4[0]
err_A_alpha_T4 = perr_alpha_T4[0]

mu_alpha_T4 = popt_alpha_T4[1]
err_mu_alpha_T4 = perr_alpha_T4[1]

sig_alpha_T4 = popt_alpha_T4[2]
err_sig_alpha_T4 = perr_alpha_T4[2]

# Messurment data used for regression Alpha
plt.errorbar(
    angle_T4[a_start:a_end],
    counts_T4[a_start:a_end],
    err_counts_T4[a_start:a_end],
    err_angle_T4,
    label='Regression Data for $K_{\\beta_2}$',
    marker = '.',
    ls = ' ',
    capsize=3.5,
    color= c.BLUE
    )

plt.plot(
    degx_alpha_T4,
    gauss(degx_alpha_T4, *popt_alpha_T4),
    label = 'Gaussan Regression $K_{\\alpha,2}$' + f'\n$A={pap.round_sig_digs(err_A_alpha_T4, A_alpha_T4)[2]}, \\mu={pap.round_sig_digs(err_mu_alpha_T4, 
    color = c.LIGHT_BLUE
    )


# Regressionfor K beta
a_start, a_end = 46, 53

p0_beta_T4_2 = [150, 12.8, 0.08]
popt_beta_T4_2, pcov_beta_T4_2 = curve_fit(gauss, angle_T4[a_start:a_end], counts_T4[a_start:a_end], p0_beta_T4_2, sigma = err_counts_T4[a_start:a_end])
degx_beta_T4_2 = np.arange(11.8, 13.8, 0.001)

perr_beta_T4_2 = np.sqrt(np.diag(pcov_beta_T4_2))

A_beta_T4_2 = popt_beta_T4_2[0]
err_A_beta_T4_2 = perr_beta_T4_2[0]

mu_beta_T4_2 = popt_beta_T4_2[1]
err_mu_beta_T4_2 = perr_beta_T4_2[1]

sig_beta_T4_2 = popt_beta_T4_2[2]
err_sig_beta_T4_2 = perr_beta_T4_2[2]

# Messurment data used for regression Alpha
plt.errorbar(
    angle_T4[a_start:a_end],
    counts_T4[a_start:a_end],
    err_counts_T4[a_start:a_end],
    err_angle_T4,
    label='Regression Data for $K_{\\beta_2}$',
    marker = '.',
    ls = ' ',
    capsize=3.5,
    color= c.ORANGE
    )

plt.plot(
    degx_beta_T4_2,
    gauss(degx_beta_T4_2, *popt_beta_T4_2),
    label = 'Gaussan Regression $K_{\\alpha,2}$' + f'\n$A={pap.round_sig_digs(err_A_beta_T4_2, A_beta_T4_2)[2]}, \\mu={pap.round_sig_digs(err_mu_beta_T4_
    color = c.YELLOW
    )

# Regressionfor K alpha 2
a_start, a_end = 56, 61

p0_alpha_T4_2 = [150, 14.5, 0.08]
popt_alpha_T4_2, pcov_alpha_T4_2 = curve_fit(gauss, angle_T4[a_start:a_end], counts_T4[a_start:a_end], p0_alpha_T4_2, sigma = err_counts_T4[a_start:a_end
degx_alpha_T4_2 = np.arange(13.8, 15.5, 0.001)

perr_alpha_T4_2 = np.sqrt(np.diag(pcov_alpha_T4_2))

A_alpha_T4_2 = popt_alpha_T4_2[0]
err_A_alpha_T4_2 = perr_alpha_T4_2[0]

mu_alpha_T4_2 = popt_alpha_T4_2[1]
err_mu_alpha_T4_2 = perr_alpha_T4_2[1]

sig_alpha_T4_2 = popt_alpha_T4_2[2]
err_sig_alpha_T4_2 = perr_alpha_T4_2[2]

# Messurment data used for regression Alpha
plt.errorbar(
    angle_T4[a_start:a_end],
```

```python
    counts_T4[a_start:a_end],
    err_counts_T4[a_start:a_end],
    err_angle_T4,
    label='Regression Data for $K_{\\beta_2}$',
    marker = '.',
    ls = ' ',
    capsize=3.5,
    color= c.DARK_PURPLE
    )

plt.plot(
    degx_alpha_T4_2,
    gauss(degx_alpha_T4_2, *popt_alpha_T4_2),
    label = 'Gaussan Regression $K_{\\alpha,2}$' + f'\n$A={pap.round_sig_digs(err_A_alpha_T4_2, A_alpha_T4_2)[2]}, \\mu={pap.round_sig_digs(err_mu_alpha_
    color = c.PURPLE
    )


plt.legend(loc = 'upper right')
plt.xlabel('Angle [deg]')
plt.ylabel('Intensity / Counts')
plt.grid(alpha = 0.6)

plt.savefig(f'{plt_save_folder}NaCl_{task}.pdf')
plt.show()
```



In [985…

```python
# extracting the according angles

lol = [1,1,2,2]
labs = ['alpha_1', 'beta_1', 'alpha_2', 'beta_2']
angles = np.array([mu_alpha_T4, mu_beta_T4, mu_alpha_T4_2, mu_beta_T4_2])
err_angles = np.array([sig_alpha_T4, sig_beta_T4, sig_alpha_T4_2, sig_beta_T4_2])

for i in range(4):
    print(f'{labs[i]}: ({pap.round_sig_digs(err_angles[i], angles[i])[2]})°')
    print(f'Expected wave length by lit value: {labs[i]}: ({pap.round_sig_digs(wlange(err_angles[i],564/2, lol[i]), wlange(angles[i],564/2, lol[i]))[2]})
    print('-' * 15)
```

```
alpha_1: (7.17 \pm 0.16)°
Expected wave length by lit value: alpha_1: (70.3 \pm 1.5)pm
---------------
beta_1: (6.3 \pm 0.3)°
Expected wave length by lit value: beta_1: (62 \pm 3)pm
---------------
alpha_2: (14.51 \pm 0.15)°
Expected wave length by lit value: alpha_2: (70.7 \pm 0.7)pm
---------------
beta_2: (12.8 \pm 0.4)°
Expected wave length by lit value: beta_2: (62.6 \pm 2.0)pm
---------------
```

In [994…
```
g_const = lam_sym / sp.sin(sym.theta) * sym.n

values = np.column_stack([
    [Mo_wavelength_alpha] * 4, [err_Mo_wavelength_alpha] * 4,
    angles * np.pi / 180, err_angles * np.pi / 180,
    [1,1,2,2]
])

temp_g = pap.do_it(g_const, [lam_sym, sym.theta, sym.n], values, [sym.n], False)

g_const = np.mean(temp_g[:,0])
err_g_const = np.mean(sqrt(temp_g[:,1]**2))
print(f'The estimated constant is: ({pap.round_sig_digs(err_g_const, g_const)[2]}) pm')
```

```
The estimated constant is: (606 \pm 19) pm
```

In [995…
```
lit_g_const_NaCl = 564.00 # pm

sd_g_each = pap.std_abw(lit_g_const_NaCl, temp_g[:,0], temp_g[:,1])
print(f'Standard diviation for each single g const:\n{sd_g_each}')

sd_g = pap.std_abw(lit_g_const_NaCl, g_const, err_g_const)
print(f'Standard diviation for the average:\n{sd_g}')
```

```
Standard diviation for each single g const:
[0.47 2.33 0.54 3.78]
Standard diviation for the average:
2.25
```

In [996…
```
# Build the dataframe
table_data = []
for i in range(4):
    row = {
        r'label': labs[i],
        r'\text{angle [deg]}': pap.round_sig_digs(err_angles[i], angles[i])[2],
        r'\text{Expected} \lambda_\text{Mo} [\mathrm{pm}]': pap.round_sig_digs(wlange(err_angles[i],564/2, lol[i]), wlange(angles[i],564/2, lol[i]))[2],
        r'\text{Lattice constant NaCl} [\mathrm{pm}]': pap.round_sig_digs(temp_g[:,1][i], temp_g[:,0][i])[2],
        r'\sigma': f'{sd_g_each[i]:.2f}'
    }
    table_data.append(row)

df_latex = pd.DataFrame(table_data)

# Add summary row for mean lattice constant
summary_row = pd.DataFrame([{
    r'label': 'Mean',
    r'\text{angle [deg]}': '',
    r'\text{Expected} \lambda_\text{Mo} [\mathrm{pm}]': '',
    r'\text{Lattice constant NaCl} [\mathrm{pm}]' : pap.round_sig_digs(err_g_const, g_const)[2],
    r'\sigma': sd_g
}])

df_final = pd.concat([df_latex.reset_index(drop=True), summary_row], ignore_index=True)

# Generate LaTeX output
print(df_final.to_latex(index=False, caption='X-ray Diffraction Analysis Results', label='tab:xray_analysis'))
```

```
\begin{table}
\caption{X-ray Diffraction Analysis Results}
\label{tab:xray_analysis}
\begin{tabular}{lllll}
\toprule
label & \text{angle [deg]} & \text{Expected} \lambda_\text{Mo} [\mathrm{pm}] & \text{Lattice constant NaCl} [\mathrm{pm}] & \sigma \\
\midrule
alpha_1 & 7.17 \pm 0.16 & 70.3 \pm 1.5 & 570 \pm 13 & 0.47 \\
beta_1 & 6.3 \pm 0.3 & 62 \pm 3 & 640 \pm 30 & 2.33 \\
alpha_2 & 14.51 \pm 0.15 & 70.7 \pm 0.7 & 567 \pm 7 & 0.54 \\
beta_2 & 12.8 \pm 0.4 & 62.6 \pm 2.0 & 641 \pm 20 & 3.78 \\
Mean &  &  & 606 \pm 19 & 2.250000 \\
\bottomrule
\end{tabular}
\end{table}
```

In [999…

```
# Calculation N_A

N_A_func = 4 * sym.M / sym.rho / (sym.g**3)

values = np.array([
    g_const * 1e-10, err_g_const * 1e-10,
    58.44,
    2.164
])

temp = pap.do_it(N_A_func, [sym.g, sym.M, sym.rho], values, [sym.M, sym.rho], False)
val_NA = temp[0][0] *1e-23
err_NA = temp[0][1] *1e-23

print(f'The Avogadro Number is: N_A = ({pap.round_sig_digs(err_NA ,val_NA)[2]}) * 10^23 1/mol')

# for the better results of the beta K lines:
values = np.array([
    temp_g[:,0][0] * 1e-10, temp_g[:,1][0] * 1e-10,
    58.44,
    2.164
])

temp = pap.do_it(N_A_func, [sym.g, sym.M, sym.rho], values, [sym.M, sym.rho], False)
val_NA_beta_1 = temp[0][0] *1e-23
err_NA_beta_1 = temp[0][1] *1e-23

print(f'The Avogadro Number for only beta_1 is: N_A = ({pap.round_sig_digs(err_NA_beta_1 ,val_NA_beta_1)[2]}) * 10^23 1/mol')
```

```
The Avogadro Number is: N_A = (4.9 \pm 0.4) * 10^23 1/mol
The Avogadro Number for only beta_1 is: N_A = (5.8 \pm 0.4) * 10^23 1/mol
```

List of Figures

List of Tables

Bibliography

[Dud26] Duden. Duden rechtschreibprüfer, 2026.

[Fin26a] Finn Zeumer. Pap 2, 2026. Zugriff am 17. Februar 2026.

[Fin26b] Finn Zeumer. Selbsterstellte python-bibliothek "papulator", 2026.

[Pro26] Proton. Lumo a.i., 2026.

[Wag25] Dr. J. Wagner. *Physikalisches Praktikum PAP 1 für Studierende der Physik*, pages 4–28. Universität Heidelberg, 2025.

[Wag26a] Dr. J. Wagner. *Physikalisches Anfängerpraktikum PAP 2.1 für Studierende der Physik*, chapter 255. Universität Heidelberg, 2026.

[Wag26b] Dr. J. Wagner. *Physikalisches Anfängerpraktikum PAP 2.2 für Studierende der Physik*, chapter 255. Universität Heidelberg, 2026.